

Lex: Turning Laws into Enforceable Security Policies

François Hublet
ETH Zürich
Zurich, Switzerland
francois.hublet@inf.ethz.ch

Jakob Merane
ETH Zürich
Zurich, Switzerland
jmerane@ethz.ch

Jonas Degelo
ETH Zürich
Zurich, Switzerland
jonas.degelo@gmail.com

Wei-En Hsieh
ETH Zürich
Zurich, Switzerland
whsieh@ethz.ch

Srdan Krstić
University of Copenhagen
Copenhagen, Denmark
srdan.krstic@di.ku.dk

David Basin
ETH Zürich
Zurich, Switzerland
basin@inf.ethz.ch

Abstract

Digital regulations such as the GDPR suffer from a major implementation gap. Compliance by Design—building software systems that satisfy legal requirements by construction—offers a promising solution, but currently lacks formal foundations and scalable tools.

We present an end-to-end, mechanized approach to Compliance by Design that translates complex digital regulations into software security mechanisms. At its core is Lex, a novel formal language tailored to the structure of legal texts. Lex supports the formalization of legal provisions and their refinement to concrete system requirements. The Lex compiler then translates the refined requirements into enforceable temporal-logic policies compatible with existing runtime enforcement and instrumentation tools.

We evaluate our approach by formalizing system-related provisions of the GDPR in about 3,200 lines of Lex code and refining this specification for two Python applications. Our methodology ensures compliance with the formalized provisions with modest runtime and development overhead. A user study provides evidence of Lex’s understandability in reading and auditing tasks for participants from legal and computer science backgrounds. To the best of our knowledge, this is the first end-to-end, mechanized, and rigorous approach for developing applications that verifiably comply with formal specifications derived from complex digital regulations.

CCS Concepts

• **Software and its engineering** → **Domain specific languages; Formal software verification**; • **Security and privacy** → **Logic and verification; Formal security models; Web application security**; • **Applied computing** → **Law**.

Keywords

Formal methods, Compliance by Design, Legal informatics

ACM Reference Format:

François Hublet, Jakob Merane, Jonas Degelo, Wei-En Hsieh, Srdan Krstić, and David Basin. 2026. Lex: Turning Laws into Enforceable Security Policies.

In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3846777>

1 Introduction

Motivation. Almost a decade after the passing of the General Data Protection Regulation (GDPR) [1], non-compliance with privacy law is still ubiquitous. In the European Union, between 80% and 95% of popular websites are estimated to violate the principle of consent-based usage of personal data, a key GDPR requirement [15]. Meanwhile, over €6B in fines have been imposed on non-compliant entities. The combination of high fines and low compliance rates implies that although companies face significant legal risk, consumers’ rights are still insufficiently guaranteed. In this context, developing practical alternatives to ex-post auditing and fines is a major challenge. One such alternative is Privacy by Design (PbD) [20], where privacy requirements are accounted for throughout the system development process to ensure privacy by construction.

Problem statement. Privacy by Design is an instance of the more general Compliance by Design (CbD) paradigm [4, 19, 60], which aims to guarantee the general regulatory compliance of systems. But while CbD is an attractive vision, in practice, (re)designing real-world systems to comply with laws such as the GDPR is difficult.

The difficulties are threefold. First, software systems and digital regulations are notoriously complex, making it hard to implement compliance safeguards correctly. Second, building compliant systems requires rigorous, interdisciplinary collaboration [81]. On the one hand, compliance can only be assessed by legal experts who rely on specific interpretative methodologies to evaluate the legal constraints applicable to a given technical system. On the other hand, assessing compliance with these legal constraints typically requires a deep understanding of the system, which only technical experts (e.g., developers) possess. Third, laws often rely on vague, abstract legal concepts, whereas system specifications are described in terms of explicit, concrete actions. To achieve CbD, a correspondence between legal concepts and system actions must thus be established.

Our solution. We provide formal foundations, a methodology, and tools for rigorous, computer-supported CbD. Our approach guarantees compliance with concrete system constraints derived through a sequence of interpretive steps from a logical formalization of an abstract regulation. This approach achieves CbD using runtime enforcement (Section 2) in the presence of complex regulations and helps interdisciplinary teams document compliance mechanisms and interpretative assumptions. Note that no formal methodology



This work is licensed under a Creative Commons Attribution 4.0 International License. *CCS '26, The Hague, Netherlands*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
<https://doi.org/10.1145/3830454.3846777>

can guarantee legal compliance in the sense of a court-proof argument, since any legal assumption made to derive system constraints must still be reviewed and validated by legal experts. However, explicit documentation of all interpretative steps can ease internal and external auditing, thereby reducing the risk of errors.

In more detail, our methodology has six steps, shown in Figure 1. It involves a legal expert/lawyer (👨) and a technical expert/developer (👩) collaborating to design a policy-compliant system. In Step 1, the lawyer and developer pair up [48] to formalize relevant parts of the law in the Lex language. Lex has a user-friendly syntax that is accessible to multidisciplinary audiences and tailored to formalizing laws. Its semantics is based on a logic with constitutive, regulative, and exception rules (Section 3). In Step 2, the experts refine this formalization by providing a *refinement mapping* between concrete system actions and legal concepts (Section 4). Specifying the formalization and refinement mappings separately allows the same formalization to be reused in different systems and makes system-specific legal assumptions explicit. In Step 3, the developer uses the Lex compiler to convert the formalization and the mapping into an equivalent policy for a Policy Decision Point (PDP). We use Metric First-Order Temporal Logic (MFOTL) [12, 24], an expressive logic supported by several existing PDPs [40, 44, 45]. In Step 4, the developer instruments the system, i.e., they instantiate a Policy Enforcement Point (PEP) by specifying how to generate all relevant events and how system actions can be caused or suppressed. This is done using the Instrlib instrumentation library [39]. In Step 5, the system is run together with the PDP and PEP, ensuring compliance with the refined formalization. Here, we use ENFGUARD [45], a state-of-the-art PDP tool for MFOTL. Finally, in Step 6, the lawyer and developer can audit the formalization, refinement, and instrumentation code, if desired. Steps 2–6 can be easily repeated when the formalization must be modified, e.g., because of a legislative change or to correct a formalization error. We give a walkthrough of our methodology by formalizing selected provisions of GDPR Articles 5–6 and refining this formalization in a microblogging application (Section 5).

Scope of guarantees. Our approach guarantees that (1) the application complies with the *formalized and refined requirements* provided that the instrumentation is sound [39] and the compiler and enforcement backends are correct. Moreover, if the Lex formalization and refinement are accepted as legally valid, our formal guarantee entails the (stronger) claim that (2) the application complies with the *formalized sections of the law* by design. We see the legal analysis of the soundness of the formalization and refinement as an independent problem that cannot be solved formally. Our methodology models legal requirements and interpretative steps but does not, on its own, establish legal compliance. In the rest of this paper, ‘compliance’ refers to system compliance with formal requirements as in (1). We assume that the developer and lawyer are honest, users cannot bypass enforcement, and our compliance tools are bug-free.

We provide evidence of our approach’s ability to (i) enforce complex law formalizations in software systems with modest performance overhead and (ii) provide a common ground for both legal and technical users to audit such formalizations (Section 6).

For (i), we present two case studies in which we enforce GDPR requirements in Python/Django applications using our methodol-

Step	What?	Who?	How?
1	Formalize the law		Lex compiler
2	Refine the formalization		Lex compiler
3	Compile the formalization		Lex compiler
4	Instrument the system		Instrlib [39]
5	Run the system and enforce		ENFGUARD [45]
6	Audit the formalization		Lex compiler

Figure 1: The Lex methodology for Compliance by Design

ogy and tools. Our 3,200-line GDPR formalization covers a broad set of system-relevant aspects of the law and is refined into specifications for two concrete applications: a microblogging service, which uses our full GDPR formalization, and a file-system manager, where enforcement focuses on a substantial core fragment (Articles 5, 6, 9, 12, 15–17, 21, 30). To the best of our knowledge, our approach is the first to enforce such an extensive set of GDPR requirements in software. We find that both applications can be instrumented for compliance with our formalization while incurring modest user-side latency and maintaining a clear separation of concerns between compliance and functional code.

For (ii), we conduct an initial user study with a group of 23 law and computer-science students to assess the understandability of Lex formalizations. Our study provides evidence that Lex formalizations are understandable by both audiences; in an auditing task, 6 of 11 interdisciplinary pairs identified a known error in a previously validated GDPR formalization.

Lastly, we review the limitations of our approach in terms of expressiveness, understandability, and performance (Section 7).

In summary, our contributions are as follows:

- We propose a new methodology for formalizing, refining, and enforcing legal specifications in software.
- We develop Lex, a new language to formalize laws, refine the legal formalization to specific systems, and compile it to enforceable temporal logic formulae.
- We provide evidence of the scalability of our approach by enforcing a large GDPR formalization in two applications.
- We provide initial evidence of Lex’s understandability to mixed legal and technical audiences through a user study.

To the best of our knowledge, this paper proposes the first end-to-end, tool-supported methodology that bridges the abstraction gap between law and policy-compliant software systems (Section 8), thus paving the way to large-scale Cbd applications (Section 9).

2 Preliminaries

We first provide background on the formal structure of laws (Section 2.1) and then introduce runtime enforcement (Section 2.2).

2.1 Formal structure of legal norms

Laws introduce both *concepts* that capture facts and *rules* that describe combinations of facts as obligatory, permitted, prohibited, etc. Formally, concepts can be seen as first-order predicates that together constitute a *legal ontology* [83]. Rules can be formalized as first-order expressions using the predicates from this ontology.

Following Searle [74], rules can be classified into two groups: *regulative* and *constitutive* rules. Regulative rules govern subjects' behavior by describing combinations of facts that shall, shall not, may, or must not occur. The two most common kinds of regulative rules are *obligation* ('shall') and *permission* ('may') rules. In contrast, constitutive rules state that a certain combination of facts 'counts as' some other fact [35]. Derived facts defined by constitutive rules are called *institutional* [74], while those facts that are not derived using rules, but instead make up 'the furniture of our physical (or physical and mental) world,' [56] are called *brute facts*.

EXAMPLE 1. Consider an ontology consisting of predicates that encode, respectively: 'the personal data d of user u is processed for the purpose p ,' 'the processing of personal data d of user u for purpose p is lawful,' and 'the user u gives consent to the system to process their personal data for the purpose p .' The rule R_1 : 'at any time, processing some personal data d of a user u for a purpose p must be lawful' is a regulative rule, since it introduces an obligation. The rule R_2 : 'when user u has given consent to the system to process their personal data for the purpose p , the processing of personal data d of user u for purpose p shall be regarded as lawful' is a constitutive rule that provides a sufficient condition for lawfulness. Processing and consent are brute facts, whereas lawfulness is an institutional fact.

Most regulative and constitutive rules contained in laws are *conditional*: obligations are applicable, and facts are constituted, only under specific circumstances [38]. However, the conditions found in law are not always necessary or sufficient. Instead, laws often use a default-exceptions structure where a general case is first stated, and exceptions are afterwards listed [38]. This suggests a formal model in which laws comprise three, rather than two, groups of rules: regulative rules, constitutive rules, and *exception* rules that specify when other rules do not apply. Note that permissions can be encoded as exceptions to obligation rules [13]; as a result, having constitutive, obligation, and exception rules is generally sufficient to formalize laws containing both obligations and permissions. Moreover, laws can contain multiple constitutive rules, in any order, that specify different combinations of facts that all 'count as' instances of the same fact. Hence, in law, unlike in traditional programming, the whole text of a law, rather than just earlier sections and explicit dependencies, must be analyzed to determine whether a given rule applies.

2.2 Runtime enforcement

We now introduce real-time, first-order enforcement (Section 2.2.1) and describe our policy language, MFOTL (Section 2.2.2).

2.2.1 Real-time, first-order enforcement. Let $\mathbb{E}$ be a set of event names, $\iota : \mathbb{E} \rightarrow \mathbb{N}$ an arity function, and $\mathbb{D}$ a set of domain values. An event $e(d_1, \dots, d_{\iota(e)})$ is composed of an event name $e \in \mathbb{E}$ and $\iota(e)$ arguments $d_1, \dots, d_{\iota(e)} \in \mathbb{D}$. A database $D \in \mathbb{DB}$ is a finite set of events and a trace $\sigma \in \mathbb{T}$ is a sequence $\langle (\tau_i, D_i) \rangle_{0 \leq i < k}$, $k \in \mathbb{N} \cup \{\infty\}$ of timestamps $\tau_i \in \mathbb{N}$ and databases $D_i \in \mathbb{DB}$, where timestamps grow monotonically ($\forall i < |\sigma|$. $\tau_i \leq \tau_{i+1}$) and progress (if $|\sigma| = \infty$, then $\lim_{i \rightarrow \infty} \tau_i = \infty$). The empty trace is denoted ϵ . An index $i < |\sigma|$ in a trace σ is called a *time-point*. A *property* is a set of infinite traces.

Fix $\mathbb{E}$. Consider a labeled transition system $S = (\mathcal{S}, s_0, \xrightarrow{\cdot})$ where $\mathcal{S}$ is a set of states, $s_0 \in \mathcal{S}$ is an initial state, and $\xrightarrow{\cdot} \in \mathcal{S} \times \mathbb{DB} \times \mathcal{S}$. Assume that $\mathbb{E}$ contains a distinguished event tick that denotes a

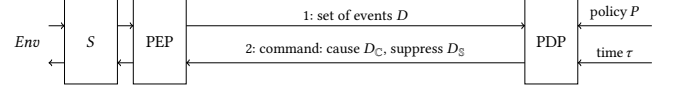


Figure 2: System model for runtime enforcement

clock tick (i.e., a timestamp increment). Given an execution $s_0 \xrightarrow{D_1} s_1 \xrightarrow{D_2} s_2 \dots$ of S , we define trace prefixes $\sigma_0, \sigma_1, \dots$ and timestamps $\tau_0, \tau_1, \dots$ as follows, where $\cdot$ denotes concatenation:

$$(\sigma_0, \tau_0) := (\epsilon, 0)$$

$$\forall i \geq 1. \sigma_i := \sigma_{i-1} \cdot \langle (\tau_{i-1}, D_i \setminus \{\text{tick}\}) \rangle$$

$$\forall i \geq 1. \tau_i := \tau_{i-1} + (\text{if tick} \in D_i \text{ then } 1 \text{ else } 0).$$

The trace of an infinite execution is $\sigma = \lim_{i \rightarrow \infty} \sigma_i$.

The system model for runtime enforcement is shown in Figure 2. The PEP and PDP must ensure that the sequence of actions executed by the system S , which communicates with an unobserved environment Env , complies with a policy P . The PDP observes the events occurring in S and generates commands, instructing the PEP to cause or suppress the actions corresponding to specific events [39]. When, as in this paper, the PDP can insert events before a clock tick, the resulting enforcement model is called *proactive* [11, 44], enabling the PDP to enforce policies with deadlines.

Which policies are enforceable crucially depends on the PEP's capabilities, i.e., which events the PEP can observe, cause, and suppress in the system. We encode capabilities in a mapping $\kappa : \mathbb{E} \rightarrow \mathbb{K}$, where $\mathbb{K}$ is a semilattice $(\mathbb{K}, \sqsubseteq)$. As suggested by Hublet et al. [44], we distinguish between events that can be only observed ($\mathbb{O}$), events that can be caused ($\mathbb{C}$) or suppressed ($\mathbb{S}$), and events that can be caused and suppressed ($\mathbb{CS}$). The operator $\sqsubseteq$ is the smallest transitive and reflexive relation satisfying $\mathbb{O} \sqsubseteq \mathbb{S}$, $\mathbb{C}$ and $\mathbb{C}, \mathbb{S} \sqsubseteq \mathbb{CS}$. Clock ticks are assumed to be only observable, i.e., $\kappa(\text{tick}) = \mathbb{O}$.

EXAMPLE 2. The predicates in Example 1 are respectively formalized as $\text{Process}(d, u, p)$, $\text{IsLawful}(d, u, p)$, and $\text{GiveConsent}(u, p)$, i.e., $\mathbb{E} = \{\text{Process}, \text{IsLawful}, \text{GiveConsent}\}$ and $\iota = \{\text{Process} \mapsto 3, \text{IsLawful} \mapsto 3, \text{GiveConsent} \mapsto 2\}$. The event $\text{Process}(d, u, p)$ reads 'the personal data d of user u is processed for purpose p ,' the event $\text{IsLawful}(d, u, p)$ denotes 'the processing of personal data d of user u for purpose p is lawful,' and the event GiveConsent stands for 'the user u gives consent to process their data for purpose p .'

Consider a web analytics portal used by a company's marketing director to list all past customer orders (state all) and view a given order (state $\text{order } i$, where i is the order's ID). The marketing director can always return to the home page. In another interface, users give consent for their data to be used for marketing purposes. The client who submitted order i is denoted $\text{Client}(i)$. Using the signature above and $s_0 = \text{all}$, the system's transitions are as follows:

$$\begin{aligned} & \text{all} \xrightarrow{\{\text{GiveConsent}(u, \text{marketing})\}} s, \\ & s \xrightarrow{\{\text{Process}(\text{order}_i, \text{Client}(i), \text{marketing})\}} \text{order } i, \quad \text{order } i \xrightarrow{0} \text{all}. \end{aligned}$$

These encode a user giving consent; moving from 'all orders' to an order; and moving back to 'all orders.' Consider the policy "whenever

$$\begin{aligned}
& \llbracket c \rrbracket_v = c, \quad \llbracket x \rrbracket_v = v(x), \quad \llbracket f(t_1, \dots, t_{a(f)}) \rrbracket_v = \hat{f}(\llbracket t_1 \rrbracket_v, \dots, \llbracket t_{a(f)} \rrbracket_v) \\
& v, i \models e(t_1, \dots, t_{a(e)}) \quad \text{iff } e(\llbracket t_1 \rrbracket_v, \dots, \llbracket t_{a(e)} \rrbracket_v) \in D_i \\
& v, i \models t \approx c \quad \text{iff } \llbracket t \rrbracket_v = c \\
& v, i \models \exists x. \phi \quad \text{iff } v[x \mapsto d], i \models \phi \text{ for some } d \in \mathbb{D} \\
& v, i \models \neg \phi \quad \text{iff } v, i \not\models \phi \\
& v, i \models \phi \wedge \psi \quad \text{iff } v, i \models \phi \text{ and } v, i \models \psi \\
& v, i \models \phi \cup_I \psi \quad \text{iff } v, j \models \psi \text{ for } j \geq i \text{ with } \tau_j - \tau_i \in I \text{ and } v, k \models \phi \text{ for all } k \in [i, j) \\
& v, i \models \phi S_I \psi \quad \text{iff } v, j \models \psi \text{ for } j \leq i \text{ with } \tau_i - \tau_j \in I \text{ and } v, k \models \phi \text{ for all } k \in (j, i]
\end{aligned}$$
Figure 3: MFOTL semantics for a fixed, infinite trace σ

some Process(d, u, p) occurs, then GiveConsent(u, p) must have previously occurred.” If $\kappa(\text{Process}) = \mathbb{S}$, i.e., data processing is suppressable, a sound PDP would typically prevent displaying an order i whenever user Client(i) has not previously consented to marketing.

2.2.2 Metric First-Order Temporal Logic (MFOTL). Let Var be a set of variables. A *signature* is a quadruple $\Sigma = (\mathbb{D}, \mathbb{E}, \mathbb{F}, \iota)$, where $\mathbb{D}$ is a set of domain values; $\mathbb{E}$ is a set of event names; $\mathbb{F}$ is a set of function symbols, where each $f \in \mathbb{F}$ has an associated function $\hat{f}$; and $\iota : \mathbb{E} \cup \mathbb{F} \rightarrow \mathbb{N}$ is an arity function.

EXAMPLE 3. *The trace*

$\sigma_1 = ((0, \{\text{GiveConsent}(A, \text{marketing})\}), (1, \{\text{Process}(d, A, \text{marketing})\}))$

describes a system execution where user A consents to their data being used for marketing at time 0, and data d of user A is processed for marketing at time 1.

In this work, we will use Metric First-Order Temporal Logic (MFOTL) [12, 24] as a policy language. Consider the following MFOTL syntax with function applications and let bindings [88]:

$$\begin{aligned}
t &::= c \mid x \mid f(t, \dots, t) \\
\phi &::= \top \mid e(t, \dots, t) \mid t \approx t \mid \neg \phi \mid \phi \wedge \phi \mid \exists x. \phi \\
&\quad \mid \phi \cup_I \phi \mid \phi S_I \phi \mid \text{let } e(x, \dots, x) = \phi \text{ in } \phi,
\end{aligned}$$

where $c \in \mathbb{D}$, $x \in \text{Var}$, $f \in \mathbb{F}$, $e \in \mathbb{E}$, and $I \in \mathbb{I}$ ranges over non-empty intervals in $\mathbb{N}$. We use standard abbreviations $\perp := \neg \top$, $\phi \rightarrow \psi := \neg \phi \vee \psi$, $\phi \leftrightarrow \psi := (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi)$, $\forall x. \phi := \neg(\exists x. \neg \phi)$, $\Diamond_I \phi := \top \cup_I \phi$ (eventually), $\blacklozenge_I \phi := \top S_I \phi$ (once), $\Box_I \phi := \neg \Diamond_I \neg \phi$ (always), and omit $[0, \infty)$ intervals from the subscript of temporal operators. We write $\bar{x}$ and $\bar{t}$ for sequences of domain variables or terms.

EXAMPLE 4. *The following formula formalizes rule R1 from Example 1 using the predicates from Example 2*

$$\Box(\forall d, u, p. \text{Process}(d, u, p) \rightarrow \text{IsLawful}(d, u, p)).$$

Given a valuation $v : \text{Var} \rightarrow \mathbb{D}$, we define the evaluation of a term t as $\llbracket t \rrbracket_v$ in Figure 3 (top). The MFOTL formula ϕ being satisfied at time-point i of trace σ under valuation v is denoted as $v, i \models \sigma \phi$. The relation $\models$ is inductively defined in Figure 3. We omit σ whenever it is clear from the context. The semantics of let is defined using a standard unrolling definition [88]. If ϕ is closed, we write $\sigma \models \phi$ to denote $\forall v, i. v, i \models \sigma \phi$.

2.2.3 MFOTL enforcement. A *specification* is a triple $S = (\Sigma, \phi, \kappa)$, where Σ is a signature, ϕ is a closed MFOTL formula over Σ , and $\kappa : \mathbb{E} \rightarrow \mathbb{K}$ gives the PEP’s capabilities. We say that S is *enforceable*

iff it is possible to make any trace σ satisfy ϕ by suppressing events $e(d_1, \dots, d_{\iota(e)})$ in σ such that $\mathbb{S} \sqsubseteq \kappa(e)$ and causing additional events $e(d_1, \dots, d_{\iota(e)})$ such that $\mathbb{C} \sqsubseteq \kappa(e)$.

Hublet et al. [45] describe a PDP algorithm and a tool, called ENFGUARD, that enforce an expressive family of MFOTL specifications. They syntactically define a fragment of MFOTL, called EMFOTL, that is enforceable by their algorithm. The EMFOTL fragment is defined using type judgments of the form $\Gamma \vdash \phi : \tau$, where $\Gamma : \mathbb{E} \rightarrow \{\mathbb{S}, \mathbb{C}\}$ and $\tau \in \{\mathbb{S}, \mathbb{C}\}$. The judgment $\Gamma \vdash \phi : \mathbb{C}$ means ‘the formula ϕ can be satisfied by causing events mapped to $\mathbb{C}$ in Γ and suppressing events mapped to $\mathbb{S}$ in Γ ’. The judgment $\Gamma \vdash \phi : \mathbb{S}$ means ‘the formula ϕ can be made false’ under the same conditions on Γ . EMFOTL contains all formulae ϕ such that $\Gamma \vdash \phi : \mathbb{C}$.

EXAMPLE 5. *With the type system from [45], the formula from Example 4 is typed to $\mathbb{C}$ under $\Gamma \equiv \text{Process} \mapsto \mathbb{S}$. Hence this formula is in EMFOTL. This matches the intuition that ϕ can be satisfied at all times by suppressing Process whenever necessary, i.e., whenever the processing is not lawful.*

3 Enforcing CRE Specifications

We now introduce a new logic, called CRE, that provides natural semantics for reasoning with constitutive, regulative, and exception rules. CRE is parametric, supporting requirements written in various object logics. We then show how, under some well-formedness conditions, CRE[MFOTL] specifications (i.e., CRE specifications with requirements written in MFOTL as the object logic) can be compiled into equivalent MFOTL specifications. Proofs of all theorems are given in the extended version of this paper [46].

3.1 The CRE logic

Constitutive rules declare certain facts as ‘counting as’ instances of other facts. The facts defined by constitutive rules are called institutional facts, while facts describing the system’s actions are called brute facts (cf. Section 2.1). In CRE, we encode institutional facts as *internal predicates* $\text{Pr}(\mathbb{E}') = \{e(\bar{x}) \mid e \in \mathbb{E}', |\bar{x}| = \iota'(e)\}$, where $\mathbb{E}'$ is a fresh set of event names disjoint from $\mathbb{E}$ and $\iota' : \mathbb{E}' \rightarrow \mathbb{N}$ is an arity function. The set of events $\mathbb{E}$ with arity function ι is reserved for brute facts. Additionally, CRE uses a set of *rule labels* $\mathcal{R}$ to identify rules and specify which rules are affected by exceptions.

Let $\mathcal{F}$ be the set of formulae over $\Sigma' = (\mathbb{D}, \mathbb{E} \cup \mathbb{E}', \mathbb{F}, \iota \cup \iota')$. A *CRE policy* is a triple $\Phi = (C, R, E)$ such that:

- C is a set of *constitutive* rules of the form $r : \phi \Rightarrow p$, where $r \in \mathcal{R}$, $\phi \in \mathcal{F}$, and $p \in \text{Pr}(\mathbb{E}')$, reading ‘ ϕ counts as p ’;
- R is a set of *regulative* rules of the form $r : \phi \Rightarrow \psi$, where $r \in \mathcal{R}$ and $\phi, \psi \in \mathcal{F}$, reading ‘if ϕ holds, then ψ shall hold’;
- E is a set of *exception* rules of the form $r : \phi \Rightarrow \text{exc}(s)$, where $r, s \in \mathcal{R}$ and $\phi \in \mathcal{F}$, reading ‘if ϕ holds, then rule s shall not apply’; and
- Each label is used at most once before ‘:’ in all of $C \cup R \cup E$.

CRE’s regulative rules are all *obligation rules*. Permissions can be formalized by combining obligations and exceptions [13].

Assume that the formulae $\mathcal{F}$ are associated with a logic X whose semantics is captured by judgments of the form $\gamma \models \sigma \phi$, where γ is a context, σ is a structure, and $\phi \in \mathcal{F}$. The logic CRE[X] extends this semantics with the three rules in Figure 4, expressed as judgments

[CON]	$\gamma \models_{\sigma}^{\Phi} p$	iff $\exists r, \phi. r : \phi \Rightarrow p \in C$ and $\gamma \models_{\sigma}^{\Phi} \phi \wedge \neg \text{exc}(r)$
[REG]	$\gamma \models_{\sigma}^{\Phi} \text{comply}(r)$	iff $\exists \phi, \psi. r : \phi \Rightarrow \psi \in R$ and $\gamma \models_{\sigma}^{\Phi} (\phi \rightarrow \psi) \vee \text{exc}(r)$
[EXC]	$\gamma \models_{\sigma}^{\Phi} \text{exc}(s)$	iff $\exists r, \phi. r : \phi \Rightarrow \text{exc}(s) \in E$ and $\gamma \models_{\sigma}^{\Phi} \phi \wedge \neg \text{exc}(r)$

Figure 4: Semantics of CRE

$\gamma \models_{\sigma}^{\Phi} \phi$. This construction can be applied to various object logics. For propositional logic, the context γ is empty and σ is a set of propositions. For first-order logic, the context γ is a valuation and σ is a first-order structure. Here, for MFOTL, the context γ is a pair v, i of a valuation and time-point and σ is a trace. We call the resulting logic CRE[MFOTL].

As above, we omit the subscript of $\models$ if there is no ambiguity. Given a constitutive rule $r : \phi \Rightarrow p$, the CON case in Figure 4 states that p holds under γ if ϕ holds *and* no exception to rule r is activated. Given a regulative rule $r : \phi \Rightarrow \psi$, REG states that r is complied with under γ , denoted $\gamma \models_{\sigma}^{\Phi} \text{comply}(r)$, if either $\phi \rightarrow \psi$ holds under γ or an exception to r is activated. Lastly, given an exception rule $r : \phi \Rightarrow \text{exc}(s)$, EXC activates an exception to s , denoted $\gamma \models_{\sigma}^{\Phi} \text{exc}(s)$, if ϕ holds under γ and no exception to r is activated. A CRE policy $\Phi = (C, R, E)$ is *complied with*, denoted $\gamma \models \Phi$, when all of its regulative rules are complied with, i.e., $\forall r : \phi \Rightarrow \psi \in R. \gamma \models_{\sigma}^{\Phi} \text{comply}(r)$.

EXAMPLE 6. Consider a CRE policy Φ with the following rules:

r1:	$\blacklozenge \text{GiveConsent}(u, p) \Rightarrow \text{IsLawful}(d, u, p) \in C$
r2:	$\text{Process}(d, u, p) \Rightarrow \text{IsLawful}(d, u, p) \in R$
r3:	$\text{JudicialOrder}(u) \Rightarrow \text{exc}(r2) \in E$

Rules r1 and r2 formalize the two rules in Example 1. Rule r3 states that if a judicial order to access u 's data exists, then rule r2 no longer applies. Now, consider these traces:

$$\begin{aligned} \sigma_1 &= ((0, \{\text{GiveConsent}(A, \text{marketing})\}), (1, \{\text{Process}(d, A, \text{marketing})\})) \\ \sigma_2 &= ((0, \emptyset), (1, \{\text{Process}(d, A, \text{marketing}), \text{JudicialOrder}(A)\})). \end{aligned}$$

Both σ_1 and σ_2 comply with Φ at time-point 1 under $v = \{d \mapsto d, u \mapsto A, p \mapsto \text{marketing}\}$. In both cases, the only regulative rule to be complied with is r2. Compliance with r2 is achieved when either $\text{Process}(d, u, p) \rightarrow \text{IsLawful}(d, u, p)$ holds or an exception to r2 is activated. The latter is the case iff $\text{JudicialOrder}(u)$ holds.

In σ_1 , we have $v, 1 \models_{\sigma_1} \text{Process}(d, u, p)$. Moreover, as $v, 0 \models_{\sigma_1} \text{GiveConsent}(u, p)$, we have $v, 1 \models_{\sigma_1} \blacklozenge \text{GiveConsent}(u, p)$. We use the constitutive rule r1, which has the premise $\blacklozenge \text{GiveConsent}(u, p)$, to establish $v, 1 \models_{\sigma_1}^{\Phi} \text{IsLawful}(d, u, p)$. We therefore have $v, 1 \models_{\sigma_1}^{\Phi} \text{Process}(d, u, p) \rightarrow \text{IsLawful}(d, u, p)$, and hence $v, 1 \models_{\sigma_1}^{\Phi} \text{comply}(r2)$. In σ_2 , we have $v, 1 \models_{\sigma_2} \text{JudicialOrder}(u)$. Using the exception rule r3, we get $v, 1 \models_{\sigma_2}^{\Phi} \text{exc}(r2)$; therefore $v, 1 \models_{\sigma_2}^{\Phi} \text{comply}(r2)$.

As in Merigoux et al. [61], we define a dependency relation $\triangleright_{\Phi}$ between constitutive and exception rules, where $r \triangleright_{\Phi} s$ iff either some $r : \phi \Rightarrow p \in C$ and p occurs in the left-hand side of the rule labeled by s , or some $r : \phi \Rightarrow \text{exc}(s) \in E$. A policy is *acyclic* if $\triangleright_{\Phi}$ has no cycles. Acyclicity excludes circular or self-referential rules and is required to compile CRE[MFOTL] to MFOTL.

3.2 Compiling CRE[MFOTL] to MFOTL

Any acyclic CRE[MFOTL] specification can be compiled into an equivalent MFOTL specification by introducing fresh exception predicates and let bindings.

Algorithm 1 CRE[MFOTL] to MFOTL compilation

```

1: Input:  $S = (\Sigma, \Phi, \kappa)$  with  $\Sigma = (\mathbb{D}, \mathbb{E}, \mathbb{R}, \iota)$ ; acyclic  $\Phi = (C, R, E)$ 
2:  $I \leftarrow \{s \mapsto e_s(\bar{x}_{\text{exc}(s)}) \mid r : \phi \Rightarrow \text{exc}(s) \in E\}$ 
3:  $R' \leftarrow \{r : \phi \wedge \neg I[r] \Rightarrow \psi \mid r : \phi \Rightarrow \psi \in R, r \in I\} \cup \{r : \phi \Rightarrow \psi \in R \mid r \notin I\}$ 
4:  $C_E \leftarrow \{r : \phi \Rightarrow I[s] \mid r : \phi \Rightarrow \text{exc}(s) \in E\}$ 
5:  $\mathbb{E}_0 \leftarrow \{p \mid p \notin \mathbb{E} \wedge p \text{ occurs in } C \cup R \cup E\} \setminus \{p \mid r : \phi \Rightarrow p \in C\}$ 
6:  $C_0 \leftarrow \{z_p : \perp \Rightarrow p \mid p \in \mathbb{E}_0\}$ 
7:  $C' \leftarrow C \cup C_E \cup C_0$ 
8:  $C' \leftarrow \{r : \phi \wedge \neg I[r] \Rightarrow p \mid r : \phi \Rightarrow p \in C', r \in I\} \cup \{r : \phi \Rightarrow p \in C' \mid r \notin I\}$ 
9:  $[(\phi_1, p_1), \dots, (\phi_k, p_k)] \leftarrow \text{collect}(\text{topological\_sort}_{\triangleright_{\Phi}}(C'))$ 
10: for  $r : \phi \Rightarrow \psi \in R'$  do  $\bar{x}_r \leftarrow \text{free\_variables}(\phi \rightarrow \psi)$  endfor
11:  $\phi \leftarrow \text{let } p_1 = \phi_1 \text{ in } \dots \text{let } p_k = \phi_k \text{ in } \bigwedge_{r : \phi \Rightarrow \psi \in R'} (\forall \bar{x}_r. \phi \rightarrow \psi)$ 
12: Output:  $S' = (\Sigma', \phi, \kappa)$  where  $\Sigma' = (\mathbb{D}, \mathbb{E} \setminus \{p_i \mid 1 \leq i \leq k\}, \mathbb{R}, \iota)$ 

```

Algorithm 2 Auxiliary function collect

```

1: function collect( $L$ )
2:    $P, L' \leftarrow \emptyset, []$ 
3:   for  $r : \phi \Rightarrow p \in L$  do
4:     if  $p \notin P$  then  $(P, L') \leftarrow (P \cup \{p\}, L' \cdot [(\bigvee_{r : \phi \Rightarrow p \in L} \phi, p)])$  endif
5:   endfor
6:   return  $L'$ 
7: end function

```

Our compilation algorithm is given in Algorithms 1–2. It assumes that (1) all constitutive rules in Φ have been normalized so that, for each internal $e \in \mathbb{E}'$, all rules constituting some $p = e(\bar{x})$ have a fixed sequence of variables $\bar{x} = \bar{x}_e$, and (2) all exception rules constituting some $\text{exc}(s)$ have a fixed sequence of variables $\bar{x} = \bar{x}_{\text{exc}(s)}$ that is a subset of the free variables of s . Algorithm 1 first creates a fresh predicate $I[r] = e_r(\bar{x}_{\text{exc}(r)})$ for any rule r that has at least one exception (l. 2). It then inserts $\neg I[r]$ in the left-hand side of rule r (l. 3) and replaces $\text{exc}(s)$ by $I[s]$ in the right-hand side of exception rules, turning them into constitutive rules (l. 3–4). All internal predicates that are never constituted in Φ are collected (l. 5) and a new constitutive rule $z_p : \perp \Rightarrow p$ is created for each of them (l. 6). Next, all constitutive rules are collected and negated exception predicates are inserted where needed (l. 7–8). The algorithm then topologically sorts the rules with respect to the order $\triangleright_{\Phi}$ on rule labels and merges these rules $r_1 : \phi_1 \Rightarrow p, \dots, r_k : \phi_k \Rightarrow p$ that define the same predicate p into a single pair $(\phi_1 \vee \dots \vee \phi_k, p)$ (Algorithm 1, l. 9 and Algorithm 2). Finally, it compiles every pair (ϕ, p) into a let binding $\text{let } p = \phi \text{ in } \dots$, preserving the same order as before, and compiles the set of (rewritten) regulative rules into a conjunction of implications, where the rule $r : \phi \Rightarrow \psi$ is turned into a closed $\forall \bar{x}. \phi \rightarrow \psi$ (Algorithm 1, l. 11). This translation is sound:

THEOREM 1. Fix $S = (\Sigma, \Phi, \kappa)$, where Φ is acyclic. Let $S' = (\Sigma', \phi, \kappa)$ be the output of Algorithm 1. Then, for any valuation v and $i \in \mathbb{N}$, we have $v, i \models \phi \iff v, i \models \Phi$.

EXAMPLE 7. The specification Φ in Example 6 translates to

```

 $\phi = \text{let } \text{IsLawful}(d, u, p) = \blacklozenge \text{GiveConsent}(u, p) \text{ in}$ 
 $\text{let } e_{r2}(d, u, p) = \text{JudicialOrder}(u) \text{ in}$ 
 $\square(\forall d, u, p. \text{Process}(d, u, p) \wedge \neg e_{r2}(d, u, p) \rightarrow \text{IsLawful}(d, u, p)),$ 

```

using the topological order r1, r3, r2.

3.3 Enforceability of CRE[MFOTL] policies

The typing rules shown in Figure 5 extend the enforceability type system for MFOTL [45] (see Section 2.2.3) to an enforceability type

$$\begin{array}{c}
\frac{r : \phi \Rightarrow \psi \in R \quad \bar{x} = \text{fv}(\phi) \cup \text{fv}(\psi) \quad \Gamma \vdash^\Phi \forall \bar{x}. \phi \rightarrow \psi : \mathbb{C}}{\Gamma \vdash^\Phi \text{comply}(r) : \mathbb{C}} \text{REGI} \\
\frac{r : \phi \Rightarrow \psi \in R \quad \bar{x} = \text{fv}(\phi) \cup \text{fv}(\psi) \quad \Gamma \vdash^\Phi \forall \bar{x}. \phi \wedge \neg \psi \rightarrow \text{exc}(r) : \mathbb{C}}{\Gamma \vdash^\Phi \text{comply}(r) : \mathbb{C}} \text{REGE} \\
\frac{r : \phi \Rightarrow \text{exc}(s) \in E \quad \Gamma \vdash^\Phi \phi : \mathbb{C} \quad \Gamma \vdash^\Phi \text{exc}(r) : \mathbb{S}}{\Gamma \vdash^\Phi \text{exc}(s) : \mathbb{C}} \text{EXC}^{\mathbb{C}} \\
\frac{\forall r : \phi \Rightarrow \text{exc}(s) \in E. (\Gamma \vdash^\Phi \phi : \mathbb{S} \text{ or } \Gamma \vdash^\Phi \text{exc}(r) : \mathbb{C})}{\Gamma \vdash^\Phi \text{exc}(s) : \mathbb{S}} \text{EXC}^{\mathbb{S}} \\
\frac{r : \phi \Rightarrow p \in C \quad \Gamma \vdash^\Phi \phi : \mathbb{C} \quad \Gamma \vdash^\Phi \text{exc}(r) : \mathbb{S}}{\Gamma \vdash^\Phi p : \mathbb{C}} \text{CON}^{\mathbb{C}} \\
\frac{\forall r : \phi \Rightarrow p \in C. (\Gamma \vdash^\Phi \phi : \mathbb{S} \text{ or } \Gamma \vdash^\Phi \text{exc}(r) : \mathbb{C})}{\Gamma \vdash^\Phi p : \mathbb{S}} \text{CON}^{\mathbb{S}}
\end{array}$$

Figure 5: Typing rules for CRE[MFOTL] policies

system for CRE[MFOTL]. The first two rules, REGI and REGE, formalize the enforceability of a regulative rule $r : \phi \Rightarrow \psi$, denoted $\Gamma \vdash \text{comply}(r) : \mathbb{C}$. This rule is enforceable when $\phi \rightarrow \psi$ can be satisfied for all valuations at all times, i.e., when $\forall \bar{x}. \phi \rightarrow \psi$ is enforceable, where $\bar{x}$ contains all free variables of $\phi \rightarrow \psi$ (Rule REGI). Alternatively, this rule is also enforceable if, whenever $\phi \wedge \neg \psi$ holds, an exception to r can be activated. This is the case when $\forall \bar{x}. \phi \wedge \neg \psi \rightarrow \text{exc}(r)$ is enforceable (Rule REGE). The rules $\text{EXC}^{\mathbb{C}}$ and $\text{CON}^{\mathbb{C}}$ provide sufficient conditions for the causability of exceptions and predicates. Given a constitutive rule $r : \phi \Rightarrow p \in C$, the predicate p is causable if ϕ is causable and the exception $\text{exc}(r)$ is suppressable; the case of exception rules is similar. The rules $\text{EXC}^{\mathbb{S}}$ and $\text{CON}^{\mathbb{S}}$ provide sufficient conditions for the suppressability of exceptions and predicates. The predicate p is suppressable if for *any* constitutive rule $r : \phi \Rightarrow p \in C$, either ϕ is suppressable or $\text{exc}(r)$ is causable.

We define ECRE[MFOTL] as the set of all $\Phi = (C, R, E)$ in CRE[MFOTL] such that for all $r : \phi \Rightarrow \psi \in R$, $\Gamma \vdash \text{comply}(r) : \mathbb{C}$. All ECRE[MFOTL] policies compile to EMFOTL formulae:

THEOREM 2. Fix $\mathcal{S} = (\Sigma, \Phi, \kappa)$ such that $\Phi \in \text{ECRE}[\text{MFOTL}]$ and Φ is acyclic. Let $\mathcal{S}' = (\Sigma', \phi, \kappa)$ be the output of Algorithm 1. Then $\phi \in \text{EMFOTL}$.

As a corollary, both ϕ and Φ are enforceable.

EXAMPLE 8. Assuming that $\kappa = \{\text{Process} \mapsto \mathbb{S}, \text{GiveConsent} \mapsto \mathbb{O}, \text{JudicialOrder} \mapsto \mathbb{O}\}$, then the policy Φ from Example 6 is enforceable. To see this, let $\Gamma \equiv \text{Process} : \mathbb{S}$. A sequent $\Gamma \vdash^\Phi \text{comply}(r2) : \mathbb{C}$ can be derived by first applying the rule REGI as follows

$$\frac{r2 : \text{Process}(d, u, p) \Rightarrow \text{IsLawful}(d, u, p) \in R \quad \{d, u, p\} = \text{fv}(\text{Process}(d, u, p)) \cup \text{fv}(\text{IsLawful}(d, u, p))}{\Gamma \vdash^\Phi \forall d, u, p. \text{Process}(d, u, p) \rightarrow \text{IsLawful}(d, u, p) : \mathbb{C}} \text{REGI}$$

and then, as in Example 4, using the rules from [45] to get $\Gamma \vdash^\Phi \forall d, u, p. \text{Process}(d, u, p) \rightarrow \text{IsLawful}(d, u, p) : \mathbb{C}$.

4 Refining CRE Specifications

In practice, some legal concepts are underspecified. Thus, specifications must be accompanied by a mapping between *concrete* events

$$\begin{array}{c}
\phi \Downarrow^+ \psi := \phi \Rightarrow \psi \quad \phi \Downarrow^- \psi := \phi \Leftarrow \psi \\
\frac{S, \Gamma \rightsquigarrow_{\text{id}} S, \Gamma}{S, \Gamma \rightsquigarrow_{\text{id}} S, \Gamma} \text{REFL} \quad \frac{S_1, \Gamma_1 \rightsquigarrow_{\pi_1} S_2, \Gamma_2 \quad S_2, \Gamma_2 \rightsquigarrow_{\pi_2} S_3, \Gamma_3}{S_1, \Gamma_1 \rightsquigarrow_{\pi_1 \circ \pi_2} S_3, \Gamma_3} \text{TRANS} \\
C' = C \cup \{r : \phi \Rightarrow e(x_1, \dots, x_k)\} \quad \text{events}(\phi) \setminus \mathbb{E} = \{e_1, \dots, e_\ell\} \\
\mathbb{E}' = \mathbb{E} \setminus \{e\} \cup \{e_1, \dots, e_\ell\} \quad i' = i[i.e_i \mapsto k_i] \setminus \{e\} \\
\kappa' = \kappa[i.e_i \mapsto \tau_i] \setminus \{e\} \quad \Gamma' = \Gamma[i.e_i \mapsto \tau_i] \setminus \{e\} \\
\forall i. \tau_i' \sqsubseteq \tau_i \quad \Gamma' \vdash \phi : \Gamma(e) \quad \nexists r', \phi', \bar{x}'. r' : \phi' \Rightarrow e(\bar{x}') \in C \\
\pi = \text{map}_{e(x_1, \dots, x_k), \phi, \{e_1, \dots, e_\ell\}} \\
((\mathbb{D}, \mathbb{E}, \mathbb{F}, i), (C, R, E), \kappa), \Gamma \rightsquigarrow_\pi ((\mathbb{D}, \mathbb{E}', \mathbb{F}, i'), (C', R, E), \kappa'), \Gamma' \text{ REF} \\
R' = R \cup \{r : \phi' \Rightarrow \psi'\} \setminus \{r : \phi \Rightarrow \psi\} \\
\frac{\phi' \rightarrow \psi' \Rightarrow \phi \rightarrow \psi \quad \Gamma \vdash^{(C, R', E)} \text{comply}(r) : \mathbb{C}}{((\mathbb{D}, \mathbb{E}, \mathbb{F}, i), (C, R, E), \kappa), \Gamma \rightsquigarrow_{\text{id}} ((\mathbb{D}, \mathbb{E}, \mathbb{F}, i), (C, R, E), \kappa), \Gamma} \text{IMPR}
\end{array}$$

Figure 6: CRE[MFOTL] refinement rules

```

1: function mape(x1,...,xk),φ,E((τi, Di)i∈ℕ)
2:   for i ∈ ℕ do D'i ← Di ∪ {e(v(x1), ..., v(xk)) | v, i ∈ σ φ}
3:   D'i ← D'i \ {e(d̄) | ⟨d1, ..., di(e)⟩ ∈ Di(e), e ∈ E} endfor
4:   return ⟨(τi, D'i)i∈ℕ⟩
5: end function

```

Figure 7: map function used in Figure 6

describing system actions and *abstract* events capturing legal notions. The legal validity of the interpretation provided by this mapping must be checked by legal experts. This approach supports reusing the same specification in different systems with different events by providing different mappings. This is a natural case for refinement techniques [2, 18, 80] in which a concrete specification implements an abstract specification.

Following a standard notion of refinement [2], we say that a concrete specification $\mathcal{S}_{\text{con}}$ *refines* an abstract specification $\mathcal{S}_{\text{abs}}$ modulo a *refinement mapping* π from concrete to abstract traces whenever $\sigma \models \mathcal{S}_{\text{con}}$ implies $\pi(\sigma) \models \mathcal{S}_{\text{abs}}$. The refinement mapping specifies how to translate concrete events into abstract events. As *enforceable* specifications are needed for runtime enforcement, our notion of enforcement must guarantee that enforceability holds for all refinements: if $\mathcal{S}_{\text{abs}}$ is enforceable, then $\mathcal{S}_{\text{con}}$ refining $\mathcal{S}_{\text{abs}}$ must be enforceable, too. Next, we define a notion of refinement for CRE[MFOTL] specifications that achieves this.

Enforceable refinement of CRE[MFOTL]. Our notion of refinement is based on the rules in Figure 6. We write $\mathcal{S}_{\text{con}} \preceq_\pi \mathcal{S}_{\text{abs}}$, reading “ $\mathcal{S}_{\text{con}}$ refines $\mathcal{S}_{\text{abs}}$ under refinement mapping π ” iff there exist Γ_{abs} and Γ_{con} such that $\Gamma_{\text{abs}} \vdash \mathcal{S}_{\text{abs}} : \mathbb{C}$, and $\mathcal{S}_{\text{abs}}, \Gamma_{\text{abs}} \rightsquigarrow_\pi \mathcal{S}_{\text{con}}, \Gamma_{\text{con}}$ can be derived using these rules.

Our refinement rules can be divided into three groups. The first group consists of the rules REFL and TRANS, which specify the relation $\rightsquigarrow$ as reflexive and transitive. The second group contains the single rule REF, which introduces the (legal) claim that a combination of concrete events should be interpreted as an abstract event. Namely, it introduces a new constitutive rule $r : \phi \Rightarrow e(\bar{x})$, where e is an abstract event. New concrete events $e_1, \dots, e_\ell$ contained in ϕ are added to the signature together with their capabilities $\tau_1, \dots, \tau_\ell$. The typing environment Γ is updated to Γ' by setting the types of each e_i to $\tau_i' \sqsubseteq \tau_i$. This choice of Γ' must ensure that $\Gamma' \vdash \phi : \Gamma(e)$,

i.e., the capabilities of the concrete events contained in ϕ must ensure that the abstract event e retains the capability $\Gamma(e)$. For instance, if e was suppressable in the abstract specification, then ϕ must be suppressable in the concrete specification. The mapping π from concrete to abstract events performed in this step is described by the $\text{map}_{e(x_1, \dots, x_k), \phi, \{e_1, \dots, e_\ell\}}$ function in Figure 7: at each time-point, new e events are added whenever ϕ holds (l. 2) and events that occur in ϕ but not in the abstract signature are removed (l. 3).

The third group contains the rules IMP_R , IMP_C , and IMP_E used to strengthen requirements. For space reasons, the refinement rules IMP_C and IMP_E are presented in the extended version of this paper [46]. The rule IMP_R replaces a rule $r : \phi \Rightarrow \psi \in R$ by a new rule $r : \phi' \Rightarrow \psi'$ whenever $\phi' \rightarrow \psi'$ implies $\phi \rightarrow \psi$ and the new rule is enforceable. Similarly, the rules IMP_C and IMP_E allow for the strengthening of the premisses of constitutive and exception rules under conditions that guarantee the soundness of the refinement.

EXAMPLE 9. Consider the policy Φ in Example 6. Suppose we have a web app that registers users' clicks on a consent banner that reads 'I accept that my data is used for marketing.' We can encode this as an observable event $\text{ClickBanner}(u)$ and introduce the constitutive rule

$$r4 : \blacklozenge \text{ClickBanner}(u) \wedge p \approx \text{"marketing"} \Rightarrow \text{GiveConsent}(u, p) \in C$$

to encode 'a user u clicking the consent banner counts as u consenting for marketing.' This uses rule REF in Figure 6.

In contrast, suppose that our app has a non-suppressable UserInput event. Then one cannot introduce the new rule

$$r5 : \text{UserInput}(d, u, p) \Rightarrow \text{Process}(d, u, p) \in C$$

since Process was suppressable in Φ but $\text{UserInput}(d, u, p)$ cannot be suppressed. Hence, rule REF is not applicable.

The rule IMP_C , shown in the extended version of this paper [46], allows us to strengthen the rule $r1$ to express that 'lawfulness holds only when consent has been given within a day's time,' i.e.,

$$r1 : \blacklozenge_{[0,1 \text{ day}]} \text{GiveConsent}(u, p) \Rightarrow \text{IsLawful}(d, u, p) \in C.$$

This strengthening is sound because it introduces a more restrictive definition of lawfulness. When enforced under this more restrictive definition, the resulting policy implies Φ . We formalize this in the extended version of this paper [46] via a notion of policy monotonicity with respect to a predicate.

Our definition fulfills the two desired properties:

THEOREM 3. If $S_{\text{con}} \preceq_{\pi} S_{\text{abs}}$ and $\sigma \models S_{\text{con}}$, then $\pi(\sigma) \models S_{\text{abs}}$.

THEOREM 4. If $S_{\text{con}} \preceq_{\pi} S_{\text{abs}}$ and S_{abs} is enforceable, then S_{con} is enforceable.

5 Compliance by Design Using LEX

We now give a detailed account of our methodology, focusing on enforcing key GDPR requirements in a microblogging app.

Our tools are free software. The Lex compiler, implemented in OCaml, has about 12,000 lines of code and is available at [this link](#).

5.1 Threat model

We consider a developer and a lawyer collaborating to produce a compliant application. The application and the law are fixed, with the former given as a functional specification and the latter in its

Art. 5 GDPR. Principles relating to processing of personal data

(1) Personal data shall be:

- (a) processed lawfully, fairly and in a transparent manner in relation to the data subject. [...]

Art. 6 GDPR. Lawfulness of processing

(1) Processing shall be lawful only if and to the extent that at least one of the following applies:

- (a) the data subject has given consent to the processing of his or her personal data for one or more specific purposes; [...]
- (f) processing is necessary for the purposes of the legitimate interests pursued by the controller or by a third party, except where such interests are overridden by the interests or fundamental rights and freedoms of the data subject which require protection of personal data, in particular where the data subject is a child.

Figure 8: Text of Articles 5(1)(a), 6(1)(a), 6(1)(f) GDPR [1]

original natural-language form. Both the developer and the lawyer are honest. In particular, the lawyer does not promote interpretations that contradict a legal consensus, while the developer correctly and faithfully instruments the application code. The application's potential users and maintainers are deemed unable to bypass or deactivate any enforcement mechanisms. They can, however, use all documented features of the developed application.

The compiler, instrumentation library [39], PDP tool [45], and Python libraries are trusted software. Any compliance breach caused solely by bugs in these components is out of the scope of this paper.

5.2 Formalizing the law in Lex

The original text of Articles 5(1)(a), 6(1)(a), and 6(1)(f) of the GDPR is shown in Figure 8 and the corresponding Lex code in Figure 10.

5.2.1 Syntax. Lex is a specification language based on CRE[MFOTL]. It provides four main types of statements:

- **Section labels** (keywords: **law**, **article**, etc.) reflect the law's structure and are ignored by the Lex compiler.
- **Type declarations** (keyword: **type**) specify the kinds of objects that the law refers to. Types are left out in CRE[MFOTL].
- **Event declarations** (keyword: **event**) correspond to the signature Σ and capabilities κ of CRE[MFOTL]. Internal events (those defined by constitutive rules) are marked as **internal**.
- **Rule declarations** (keyword: **rule**) define rules as in CRE, using a human-friendly syntax: e.g., $r : \phi \Rightarrow \psi \in R$ becomes **rule** r **whenever** ϕ **oblige** ψ . Lex uses keywords instead of symbols (e.g., **ONCE** for $\blacklozenge$) in formulae. It also adds syntactic sugar to CRE[MFOTL] by supporting the conjunction of several formulae, predicates, or exceptions (one per line) on each side of a rule.

Figure 9 contrasts Lex and CRE[MFOTL] syntax.

5.2.2 Formalization. To formalize Article 5, we first declare types capturing data processing activities, data, data subjects, legal bases, purposes, and entities (l. 5–9 of Figure 10). The **legal_basis** and **purpose** types are subtypes of **string**. Next, we declare events. The **DataProcessing** event (l. 12–17) encodes data processing by a processor on behalf of a controller for a given purpose. It is marked as **suppressable**, e.g., the PEP can prevent data processing to avoid violations. The documentation string (docstring) in triple quotes

Lex	law lbl ["title"] article lbl ["title"] paragraph lbl... point lbl... ...	[τ] event e ""<docstring>"" $x_1 : t_1$... $x_k : t_k$	rule r whenever ϕ_1 ... ϕ_k (constitute refine) p_1 ... p_ℓ	rule r whenever ϕ_1 ... ϕ_k oblige ψ_1 ... ψ_ℓ	rule r whenever ϕ_1 ... ϕ_k except lbl_1 ... lbl_ℓ
	type t_1 [is t_2] ""<docstring>""	$\tau ::= \text{observable}$ suppressable causable internal			
CRE[MFOTL]	n.a.	$e \in \mathbb{E}$ $\iota(e) = k$ $\kappa(e) = \mathcal{O}, \mathbb{S}, \mathbb{C}, \dots$	$r_1 : \bigwedge_{i=1}^k \phi_i \Rightarrow p_1 \in \mathcal{C}$... $r_\ell : \bigwedge_{i=1}^k \phi_i \Rightarrow p_\ell \in \mathcal{C}$	$r_1 : \bigwedge_{i=1}^k \phi_i \Rightarrow \psi_1 \in \mathcal{R}$... $r_\ell : \bigwedge_{i=1}^k \phi_i \Rightarrow \psi_\ell \in \mathcal{R}$	$r_1 : \bigwedge_{i=1}^k \phi_i \Rightarrow \text{exc}(s_1) \in \mathcal{E}$... $r_\ell : \bigwedge_{i=1}^k \phi_i \Rightarrow \text{exc}(s_\ell) \in \mathcal{E}$

Figure 9: Correspondence between the Lex and CRE[MFOTL] syntax

on l. 13 describes the event’s legal meaning. The **PersonalData** event encodes that data is personal to a data subject (l. 19–22). The **IsLawful**, **IsFair**, and **IsTransparent** events denote that an activity is lawful, fair, and transparent, respectively (l. 23–34), and **HasPurpose** encodes that an activity has a given purpose (l. 42–45). **IsLawful** is marked as **internal** as Article 6 defines ‘lawfulness.’

We can now state the regulative rule in Article 5(1)(a) (l. 42–50). This rule expresses that whenever personal data is processed (l. 52–53), processing must be lawful, fair, and transparent (l. 55–57). The annotation **enforceable suppressing condition**[0] (l. 58) instructs Lex to type the rule to $\mathbb{C}$ by suppressing the first premiss, i.e., **DataProcessing**, whenever needed.

Article 6(1)(a) and 6(1)(f) give two sufficient conditions for lawfulness: user consent for all current purposes of processing (event **GiveConsent**, l. 64–68) and legitimate interest of the controller or a third party (l. 69–73). Article 6(1)(a) constitutes lawfulness when consent has been given by the data subject, in the past or present, to process their data. This translates to the constitutive rule l. 84–90. Since consent may be given at any point prior to processing, we use the temporal keyword **ONCE** before **GiveConsent** on l. 88.

Article 6(1)(f) contains an exception to processing based on “legitimate interest” when “such interests are overridden by the interests [...] of the data subject.” We formalize this by two rules. The first rule (l. 94–99) constitutes lawfulness when legitimate interest is claimed. The second rule (l. 101–106) creates an exception to the first rule when the interests of the data subject prevail (event l. 75–79).

5.3 Refining the formalization in Lex

We now refine the above specification of a GDPR fragment into a concrete specification for our target system.

5.3.1 Syntax. Lex provides the following syntax:

- *Type refinements*, declared as **refine type... is...**, map an abstract type to a concrete type.
- *Event refinements* (Rule REF in Figure 11) are declared in a similar way as constitutive rules, but using the keyword **refine** instead of **constitute**. To refine an event to $\top$ (i.e., claim it always occurs) or $\perp$ (i.e., claim it never occurs), the syntax **assume (true|false)** can be used.
- *Strengthening and weakening* (Rules IMP_C, IMP_R, IMP_E) are declared as **replace (strengthen | weaken)...** **by...** where both keywords can be followed by multiple rule labels.

5.3.2 Target application and enforcement approach. Consider a microblogging application in which users can post text messages and read each other’s posts. To enforce Articles 5(1) and 6(1)(a),(f), we can prevent reading a user’s personal data from, or writing it to, the database when the user has not consented to the current purpose of processing. Moreover, we assume that legal experts want to make the (informal) claim that fairness and transparency are always guaranteed by the system’s overall design.

Formalization. The Lex refinement code for our application is shown in Figure 11. This code is introduced by the **refine** keyword followed by the name of a Lex file (l. 1). The code’s first part (l. 3–34) declares the types and events in the application’s ontology. Note the informal docstrings attached to events, which are crucial for lawyers to assess the legal admissibility of any subsequent refinement rules.

The rest of the code (l. 36–77) describes the refinement. First, abstract types are refined to concrete types (l. 36–40). Then, the **assume true** statements l. 42–45 encode the lawyer’s informal claim and they come with informal justifications. Finally, four rules describe the mapping between concrete application events and abstract GDPR events (l. 48–77). Note that in line with the current legal consensus, legitimate interest can be invoked to justify the processing of user data without consent for the core service purpose (rule l. 65–70). In contrast, legitimate interest cannot be relied on for non-essential advertisement and analytics purposes [78]. For the latter, explicit user consent is required.

5.4 Compiling and instrumenting the system

Our compilation pipeline is shown in Figure 12. It relies on a series of intermediate representations [58]. First, the parser processes the .lex (law) and .rex (refinement) files to produce a lex/rex object. Then, the type-checker converts the untyped object produced by the parser into a typed tlex (or trex) object. It assigns a unique *label* to each rule, which encodes the hierarchy of sections in which each rule is contained, and collects the exceptions occurring in the code. The enforceability checker converts the typed object produced by the previous phase into a typed elox (or erex) object. This succeeds if every obligation rule marked as **enforceable** successfully checks for enforceability. Finally, using Algorithm 1, the compiler converts the elox object into an MFOTL-like cllex object that is then linearized into a formula. When compiling a rex file, the corresponding lex specification is typed in parallel with the refinement code. The SMT solver Z3 [26] is used to verify the strengthening rules.

```

1  law "GDPR"
2
3  article "5" "Principles relating to
4    processing of personal data"
5
6  type activity
7  type data
8  type data_subject
9  type legal_basis is string
10 type purpose is string
11 type entity
12
13 suppressable event DataProcessing
14   """Data {d} is processed by processor {p}
15   on behalf of controller {c} as part
16   of data processing activity {a}"""
17   p : entity
18   c : entity
19   a : activity
20   d : data
21
22 observable event PersonalData
23   """Data {d} is personal data of {ds}"""
24   d : data
25   ds : data_subject
26
27 internal event IsLawful
28   """Activity {a} is lawful with legal
29   basis {b}"""
30   a : activity
31   b : legal_basis
32
33 observable event IsFair
34   """Activity {a} is fair"""
35   a : activity
36
37 observable event IsTransparent
38   """Activity {a} is transparent in relation
39   to data subject {ds}"""
40   a : activity
41   ds : data_subject
42
43 observable event IsCollection
44   """Activity {a} collects personal data
45   from data subject {ds}"""
46   a : activity
47   ds : data_subject
48
49 suppressable event HasPurpose
50   """Data processing activity {a} has
51   purpose {p}"""
52   a : activity
53   p : purpose
54
55 paragraph "1"
56   point "a"
57
58 rule
59   whenever
60     DataProcessing(p, c, a, d)
61     PersonalData(d, ds)
62   oblige
63     EXISTS b. IsLawful(a, b)
64     IsFair(a)
65     IsTransparent(a, ds)
66   enforceable suppressing condition[0]
67
68 article "6" "Lawfulness of processing"
69
70 type interest
71
72 observable event GiveConsent
73   """Data subject {ds} gives consent to
74   controller {c} to use their data for
75   purpose {p}"""
76   ds : data_subject
77   p : purpose
78   c : entity
79
80 observable event IsNecessaryForLegitimateInterest
81   """Data processing activity {a} is necessary to
82   protect the interest {i} of party {e}"""
83   a : activity
84   e : entity
85   i : interest
86
87 observable event IsOverriddenByDataSubjectInterests
88   """Interest {i} of entity {e} is overridden by
89   the interests of data subject {ds}, in
90   particular when {ds} is a child"""
91   e : entity
92   i : interest
93   ds : data_subject
94
95 paragraph "1"
96   point "a"
97
98 rule
99   whenever
100     DataProcessing(p, c, a, d)
101     PersonalData(d, ds)
102     FORALL p. HasPurpose(a, p) IMPLIES ONCE
103       GiveConsent(ds, p, c)
104   constitute
105     IsLawful(a, "6(1)(a)")
106
107 point "f"
108
109 rule "legitimate_interest"
110   whenever
111     DataProcessing(p, c, a, d)
112     IsNecessaryForLegitimateInterest(a, e, i)
113   constitute
114     IsLawful(a, "6(1)(f)")
115
116 rule
117   whenever
118     PersonalData(d, ds)
119     IsOverriddenByDataSubjectInterests(e, i, ds)
120   )
121   except
122     rule "legitimate_interest"

```

Figure 10: Lex formalization of Articles 5(1)(a), 6(1)(a), 6(1)(f) GDPR from Figure 8

Instrumentation consists in (i) marking operations, such as function calls, that should trigger events and (ii) defining the PEP's logic for suppressing and causing events. The Instrlib library [39] provides instrumentation primitives for Python/Django applications and bindings for the ENFGUARD tool. Following instrumentation with Instrlib, running the Python/Django application launches the ENFGUARD tool together with the app.

5.5 Auditing the system

The Lex compiler can generate HTML documentation files showing the Lex code and a natural language reading of the formalization generated from its docstrings. The documentation can be enriched with information from Akoma Ntoso [85] and Formex [66] files, two popular machine-readable legal document standards. The HTML documents generated by the Lex compiler support auditing by legal and technical experts (see Section 6, RQ4). Similarly, the Lex refinement can be audited to confirm that the formalized interpretation of system events in terms of the law's concepts is legally sound.

6 Evaluation

We evaluate our approach by answering four research questions:

RQ1. Is Lex expressive enough to formalize legal requirements from various jurisdictions and legal domains?

RQ2. Given requirements in Lex, can one successfully enforce them in applications using our pipeline? How much engineering effort does the instrumentation require?

RQ3. What runtime overhead does our pipeline incur?

RQ4. Is Lex code understandable by both legal and technical audiences? Can legal and technical experts effectively collaborate to read and audit Lex specifications?

A user study of Lex *writing* is out of scope, as it would require extensive training of study subjects. A full assessment of our applications' compliance by external legal experts is also left for future work.

RQ1. Legal expressivity. We provide evidence of Lex's expressivity across three fragments of the EU's GDPR, the Swiss BGG, and the U.S. IRC. These norms were chosen for their prominence and heterogeneous domains and jurisdictions. The GDPR is the European Union's main privacy law and one of the world's most influential digital regulations. The BGG regulates the appeal procedures before the Swiss Federal Supreme Court. The IRC is the United States' federal tax code; the parts of Section 121 that we analyze were previously formalized in the Catala language [61].

Table 1 summarizes our case studies. To develop the specifications, we followed a pair programming approach [61] involving a computer scientist and a lawyer. The BGG and IRC fragments each

```

1 refine gdpr
2
3 type activity_id is string
4 type user_id is string
5 type data_id is string
6 # [... more type declarations omitted]
7
8 suppressable event Read
9   """Data {id}, which is personal data
   of {owner}, is read for purpose {
   purpose} in the context of activity {
   activity} while building a webpage
   for user {user}."""
10   id : data_id
11   owner : user_id
12   activity: activity_id
13   purpose : purpose
14   user : user_id
15
16 suppressable event Write
17   """Data {id} ... is written ..."""
18   id : data_id
19   owner : user_id
20   activity: activity_id
21   purpose : purpose
22   user : user_id
23
24 suppressable event Collect
25   """Data {data} ... is collected ..."""
26   activity: activity_id
27   data : data_id
28   owner : user_id
29   purpose : purpose
30
31 suppressable event Consent
32   """User {user} consents to their data
   being used for purpose {purpose}."""
33   user : user_id
34   purpose : purpose
35   # [... more event declarations omitted]
36
37 refine type activity is activity_id
38 refine type data is data_id
39 refine type data_subject is user_id
40 refine type entity is string
41 refine type interest is string
42
43 assume true IsFair
44   """The automated enforcement of the GDPR
   requirements ensures fairness with
   respect to the user. Only strictly
   necessary data is used for each purpose
   ."""
45
46 assume true IsTransparent
47   """The automated enforcement of the GDPR
   requirements, in particular the ROPA
   logging mechanism, ensures transparency
   with respect to the user."""
48
49 rule "r_DataProcessing"
50   whenever
51     (EXISTS ds'. Read(d, ds, a, p, ds'))
52     OR (EXISTS ds'. Write(d, ds, a, p, ds
53         ')) OR Collect(a, d, ds, p)
54   refine
55     DataProcessing("GDPRSocial, Inc.", "
56         GDPRSocial, Inc.", a, d)
57
58 rule "r_HasPurpose"
59   whenever
60     (EXISTS ds'. Read(d, ds, a, p, ds')) OR (
61     EXISTS ds'. Write(d, ds, a, p, ds')) OR
62     Collect(a, d, ds, p)
63   refine
64     HasPurpose(a, p)
65
66 rule "r_GiveConsent"
67   whenever
68     Consent(ds, p) OR (EXISTS sp. SpecialConsent
69     (ds, p, sp))
70   refine
71     GiveConsent(ds, p, "GDPRSocial, Inc.")
72
73 rule "r_IsNecessaryForLegitimateInterest"
74   whenever
75     (EXISTS ds'. Read(d, ds, a, p, ds')) OR (
76     EXISTS ds'. Write(d, ds, a, p, ds')) OR
77     Collect(a, d, ds, p)
78   p = "service"
79   refine
80     IsNecessaryForLegitimateInterest(a, "
81     GDPRSocial, Inc.", "Providing the service and
82     performing GDPRSocial's normal operations.")
83
84 rule "r_IsOverriddenByDataSubjectInterests"
85   whenever
86     (EXISTS ds'. Read(d, ds, a, p, ds')) OR (
87     EXISTS ds'. Write(d, ds, a, p, ds')) OR
88     Collect(a, d, ds, p)
89   NOT (p = "service")
90   refine
91     IsOverriddenByDataSubjectInterests(c, i, ds)

```

Figure 11: Lex refinement for Articles 5(1)(a), 6(1)(a), 6(1)(f) GDPR

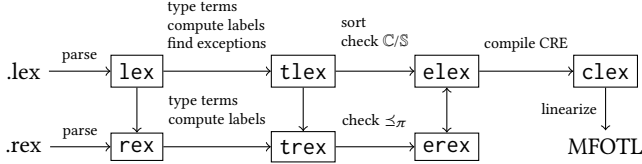


Figure 12: Structure of the Lex compiler

Law	Domain	Jurisdiction	Scope	LOC
GDPR	Privacy law	EU	5–10, 12–22, 30, 45, 46, 49	3,221
BGG	Public law	Switzerland	82(1), 83(1)(a–e)	234
IRC	Tax law	United States	121(a), 121(b)(1)	131

Table 1: RQ1: Lex formalization statistics

required less than 2 hours of work. The much larger GDPR specification was completed within 5 working days.

Our GDPR formalization targets 21 articles directly regulating system behavior, excluding, e.g., provisions establishing administrative bodies and procedures (e.g., Art 50–59, 60–76). Statistics about our formalization’s coverage are given in Table 2. The formalization consists of about 3,200 lines of code including 21 type declarations, 211 event declarations, and 203 rule declarations. While formalization is always a subjective process, we strived to make the formalization as literal as possible and found limited space for interpretation. **Overall, our case studies suggest that Lex captures heterogeneous requirements across three jurisdictions, including a large GDPR fragment.**

Status	% of law	% of scope	% of formalized
Formalized (F)	18.4%	69.7%	100.0%
Conservatively assumed (C)	7.8%	29.6%	42.5%
Enforced at runtime (E)	7.2%	27.1%	38.9%
Informally assumed (I)	6.4%	24.1%	34.6%

Table 2: GDPR coverage in GDPRSOCIAL, by word count

RQ2. Feasibility. To showcase our pipeline’s end-to-end Cbd support, we develop two Python apps: GDPRSOCIAL and GDPRFS.

GDPRSOCIAL is a microblogging service where users can publish posts, view other users’ posts, and follow each other. Users’ timelines show the posts of followed users. GDPRSOCIAL processes data for three purposes: providing the application’s core social network functionality; showing personalized advertising based on a user’s timeline; and collecting analytics data about individual pages. GDPRSOCIAL also displays privacy notices and a dashboard allowing users to exercise their data subject rights. GDPRSOCIAL’s design aims to address all formalized GDPR provisions.

GDPRFS is a FUSE-based file system [59] designed to support GDPR-compliant handling of personal data by small businesses. It guards local file operations (creation, reading, writing...) and provides two interfaces for company employees and customers whose personal data is processed. In the employee interface, employees log in and declare the purpose of their current actions. The customer interface, deployed as a web server, allows users to manage their consent and exercise their data subject rights. An LLM-based analyzer is used to flag personal data in files. The local and web components are regularly synchronized. GDPRFS’s design aims for compliance with a core set of provisions (Art 5, 6, 9, 12, 15–17, 21, 30).

Application	Baseline LOC		Instrumented LOC					Rex
	Python	HTML/CSS	Python	HTML/CSS	Lex			
GDPRSOCIAL	1,679	2,123	2,445	+766	2,123	+0	3,221	1,021
GDPRFS	–	–	2,536		1,135		1,815	639

Table 3: RQ2: Instrumented application statistics

Refinement. The refinement code for GDPRSOCIAL has about 1.0k lines of Lex code, defining 36 events, 115 assumptions, 67 rules, and 3 strengthening statements. The slightly smaller GDPRFS refinement has about 600 lines of code. We distinguish between three ways of discharging formalized provisions: informal assumptions requiring legal review (**I**, 34.6% of formalized provisions), those refined and enforced at runtime (**E**, 38.9%), and *conservative* assumptions (**C**, 42.5%). An assumption is deemed conservative when its incorrectness cannot degrade compliance. For instance, the assumption that the application never claims legitimate interest is conservative, since it only reduces the possible operations that can be performed under enforcement [39]. As a consequence, conservative assumptions do not require extensive legal review. Note that the sum total is more than 100% as certain provisions are discharged through a combination of **I**, **E**, and **C**. About two-thirds of all formalized provisions are discharged without relying on informal assumptions (i.e., via **E** or **C** but not **I**). Creating each .rex file required approximately 5 developer-days of work.

Instrumentation. Table 3 provides statistics about the instrumented applications. For GDPRSOCIAL, we also produced an uninstrumented version of the application by removing all enforcement-related code, which we refer to as the *baseline*. The baseline provides similar social functionality without any GDPR compliance guarantees. In addition to the Lex formalization and Rex refinement code, instrumenting GDPRSOCIAL required 766 additional lines of Python code, amounting to 20% of the uninstrumented code base (46% of the Python code base). Using Instrlib [39], this code is concentrated in a single enforcer.py file, thereby supporting separation of concerns. With the refinement at hand, developing and testing the full GDPRSOCIAL application required about 7 developer-days of work.

For GDPRFS, the baseline is the local file system; hence reporting baseline LOC is not possible. Instrumentation is concentrated in a single myfs.py file and has approximately 500 LOC. Developing the GDPRFS application required approximately 20 developer-days. **In summary, our case studies demonstrate how our methodology supports the end-to-end enforcement of a large formalization in two apps with separation of concerns, manageable code base growth, and reasonable development effort.**

RQ3. Runtime performance. For each application, we measured the latency of representative actions. In GDPRSOCIAL, these actions are showing the user’s timeline, creating a post, giving or revoking consent, accessing privacy notices, and downloading one’s personal data (*right to access*, Art 15 GDPR). In GDPRFS, they are reading and writing a file, giving or revoking consent, requesting erasure of personal data, and requesting rectification of inaccurate data.

We vary the number of data items (posts in GDPRSOCIAL, files in GDPRFS) for $n \in \{10^2, 10^3, 10^4\}$. When adding elements, we populate the corresponding consent entries and enforcer state, thereby

Application	Request	Baseline ($\propto n$)			Instrumented ($\propto n$)		
		10^2	10^3	10^4	10^2	10^3	10^4
GDPRSOCIAL	Timeline	61	61	63	91	+30	106 +45 102 +39
	Post	60	66	57	104	+44	103 +37 102 +45
	Consent	75	58	62	74	-1	102 +44 111 +49
	Revoke	62	56	54	85	+23	92 +36 95 +41
	Notices	72	58	68	70	-2	90 +32 76 +8
	Access	65	59	72	85	+20	75 +16 77 +5
GDPRFS	Read	<1	<1	<1	74	+74	67 +67 79 +79
	Write	<1	<1	<1	2.3s	+2.3s	2.2s +2.2s 2.2s +2.2s
	Consent	–	–	–	12		13 14
	Revoke	–	–	–	13		14 15
	Erasure	–	–	–	1.0s		1.0s 1.0s
	Rectification	–	–	–	21		22 22

Table 4: RQ3: Runtime latency (ms, unless otherwise stated). +xx denotes overhead with respect to baseline.

simulating a long-running system. Our measurements are performed on a 2.4 GHz Intel i5 with 32 GB RAM over 20 repetitions. In GDPRFS, gpt-5.4-mini is used as the analyzer.

Table 4 shows the average latencies and compares them to the baseline application without enforcement. In GDPRSOCIAL, the latency of the instrumented application always remains below 120ms, with no significant slowdown as n increases. The enforcement overhead of up to 50ms corresponds to a series of calls to ENFGUARD, each of which evaluates the compliance of 1–50 events for a very large MFOTL formula containing 6,655 operators and predicates. Overall, the latency of the instrumented GDPRSOCIAL comes close to the 100ms latency mark required for “instantaneous” interaction [62, p. 135]. In GDPRFS, the only workloads exceeding 100ms are writing and erasure. Writing invokes the LLM analyzer, which accounts for most of the latency (2.0s); erasure involves selecting and deleting a large number of files. **Excluding the cost of LLM calls in GDPRFS, we observe moderate runtime overhead (0–80ms) due to enforcement in both apps, with most workloads being near-instantaneous (<150ms).**

RQ4. Understandability. We assessed Lex’s understandability using both subjective and objective measures. Perceived usability was assessed using the System Usability Scale (SUS) [17], a technology-agnostic questionnaire designed to assess the subjective usability. SUS scores range from 0 to 100, with “OK” usability starting at a score of 52 and “good” usability at 73 [8]. Objective comprehension was evaluated through reading and auditing tasks.

Procedure. We recruited 23 students, of whom 12 had a background in law and 11 in computer science or a related technical field, from the subject pool of our institution’s behavioral research laboratory (829 computer science and 590 law students). Pool members were invited by email and registered electronically under standard conditions. All participants received USD 55 as compensation for the 2-hour session. Participant characteristics are reported in the extended version of this paper [46].

The participants were first given a 30-minute training on Lex and received printed documentation. They then solved a comprehension task in which they matched a Lex formalization to one of four natural-language options. This initial task was followed by the System Usability Scale (SUS) [17]. We adapted the SUS by referring to Lex as a “language” instead of a “system,” and rephrased

	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6
Law (n = 12)	41.7%	66.7%	83.3%	83.3%	91.7%	91.7%
CS (n = 11)	81.8%	72.7%	81.8%	72.7%	100%	100%

Table 5: Correct responses in the reading tasks, by group

two questions to reflect participants’ task of understanding rather than “using” Lex. After the SUS, participants were given a reading comprehension task consisting of six questions. Finally, the participants completed an auditing task. Answers were collected anonymously using Qualtrics. Free-text answers were analyzed only for the auditing task. One author coded each answer along two dimensions — whether the seeded mistake was identified, and whether further issues were raised — both before and after the group phase. A second author reviewed and revised the coding.

System Usability (SUS). Given the small sample, we report descriptive statistics only and refrain from null hypothesis significance testing throughout our analysis. The overall mean SUS score was 66.1 (SD = 17.3), which falls into the “OK” usability range [8]. Computer science students reported higher scores ($M = 73.9$, $SD = 13.3$, “good”) than law students ($M = 59.0$, $SD = 17.9$, “OK”). As an internal consistency check, one SUS item was presented twice. Of the 23 participants, 18 (78%) gave identical responses to both instances; the remaining 5 deviated by exactly one point on the five-point scale, and no participant deviated by more than one point (Pearson’s $r = 0.88$). Since all deviations were within one scale point and the study was conducted as a supervised in-person lab session, we found no indication of careless responding and therefore excluded no participants.

Reading. Most participants (87.0%; Law: 83.3%, CS: 90.9%) solved the initial comprehension task correctly. In the six reading tasks, correct response rates ranged from 41.7% to 91.7% among law students and from 72.7% to 100% among computer science students (see Table 5). Although computer science students reported higher perceived usability, their performance in the reading tasks was not consistently better; this difference was not tested inferentially.

Auditing. The auditing task involved reviewing a Lex formalization of Article 7(1) GDPR. This formalization was developed by Bartolini et al. [10], who validated it by asking four legal experts to answer six questions assessing its quality. The original formalization consisted of Reified I/O logic formulae [70] together with a human-readable translation, a format comparable to our Lex documentation files. The experts gave overwhelmingly positive answers to all questions [10]. However, the formalization was later found to contain a mistake in its encoding of temporal relations [43].

To assess whether participants using Lex could identify the mistake, we translated the erroneous formalization into Lex and presented the generated documentation file to our participants. First, each participant worked individually; then, 22 participants were grouped into 11 interdisciplinary pairs to collaboratively re-audit the same rule. Six out of 11 pairs (55%) identified the temporal encoding mistake in the formalization. Participants also raised further terminological or logical points (11 out of 23, 48%). Of these, 6 (26%) flagged the GiveConsent predicate as logically redundant, an anticipated but legitimate concern about the formalization. Most of the remaining points concerned Lex’s notation itself, chiefly the semantics of the **oblige** keyword and the naming of the **entity** and

controller parameters; we count these as comprehension difficulties rather than as findings about the formalization. **Overall, our user study provides initial evidence that participants from both backgrounds can read Lex formalizations after a short introduction, and that about half of the interdisciplinary pairs detected a known error in a curated auditing task.**

7 Limitations

Expressivity (RQ1). While expressive, the variant of MFOTL used in this paper has several potential limitations. First, it does not support an explicit encoding of hyperproperties [25]. In the literature, hyperproperties such as information flow have been used to formalize compliance requirements, including purpose limitation [41], data minimization [3], and erasure [47]. In this paper, information flow is handled declaratively, which avoids the high overhead of previous approaches [41, 42], but introduces additional assumptions. For instance, refinement rules simply declare a datum’s owner (e.g., as another field of the same record), rather than automating ownership identification via dynamic or static information flow tracking. Our approach could be combined with previous works [42] to add such mechanisms in the instrumentation; potential pitfalls include higher runtime costs and overconservative enforcement. Second, Lex does not currently support branching-time properties, which can be useful to reason about possible—but not certain—future system states, e.g., when refining data minimization and storage limitation requirements. Such properties are not easily enforceable at runtime and require static verification. Third, Lex does not support cyclic specifications. Cyclic specifications of legal requirements have been proposed by DeYoung et al. and operationalized as fixed points [29]. A variant of MFOTL with recursive let bindings [88] could be used to implement such an extension. Note that all these limitations are common to eFLINT [82] and Catala [61].

The presence of non-formalized requirements in our GDPR code is largely independent of these formal limitations. The vast majority of the requirements we left unenforced either did not result in specific system constraints (national overrides, informal definitions) or had no clear runtime enforcement strategy (e.g., ensuring “freely given consent”). The former were not formalized, while the latter were assumed with an informal justification in the refinement.

Feasibility/Understandability (RQ2,4). All formalization, refinement, and instrumentation code presented in this paper was developed by the authors, without external peer review. It thus bears proof of technical feasibility, not of general end-to-end usability. Furthermore, while our approach is generally language-agnostic, its applicability to legacy code bases remains to be evaluated.

Our user study focused on Lex’s understandability in reading and auditing tasks and was conducted in a controlled setting with students. Because of the small sample ($N = 23$) and the resulting low statistical power, we report descriptive statistics only. Similarly, the success of 6 out of 11 pairs in the seeded auditing test is encouraging but provides only modest evidence of auditability.

Additional work is required to validate the generality of our findings in real-world development settings. For full generality, user studies should involve (i) larger cohorts, allowing for adequately

powered statistical analysis, (ii) professional developers, compliance lawyers, and regulators rather than students, (iii) writing specifications rather than just reading them, (iv) end-to-end scenarios, exercising the whole proposed methodology, (v) formalization and refinement by multiple external interdisciplinary pairs, and (vi) compliance with both new and legacy codebases. Challenges include not only the recruitment and training of participants but also the specification of compliance goals and the development of an interdisciplinary evaluation procedure. To the best of our knowledge, there is no fully standardized methodology for validating legal formalizations. Accordingly, this paper has focused on providing a framework for enforcing formalized provisions, rather than legally validating the formalization itself.

Performance (RQ3). We measured latency in two monolithic, medium-sized applications without advanced load balancing or sophisticated middleware. Assessing performance in more complex settings, especially those involving multiple interacting software systems and non-web apps, is left for future work.

8 Related work

Compliance by Design. Few previous works propose technical approaches to CbD. To the best of our knowledge, none of them provides an end-to-end, tool-supported methodology for CbD.

Formal business process compliance methodologies have been proposed using Petri nets [60] and defeasible deontic logic [71]. While rigorous, these approaches lack a tailored formal language. Julisch et al.'s [50] informal approach enumerates control patterns for auditing the regulatory compliance of financial information systems. Their interdisciplinary work is tailored to financial regulations and does not rely on formal modeling. Krstić et al. [54] propose a model-driven privacy approach that automatically generates PEPs tailored for GDPR-style purpose limitation and consent. Recent standards for compliance with digital regulations such as ISO 31700 [49] are also based on informal requirements and examples.

Formalizability of legal norms. Since the 1980s [76], an interdisciplinary community has been studying the formalization of legal requirements. In our work, we adopted a pragmatic view, acknowledging the challenges of legal formalization while underlining its usefulness for CbD. We recognize that (i) all legal provisions display some degree of ambiguity or vagueness [7, 27, 65, 72, 79, 86]; (ii) the interpretation of legal norms varies depending on legal practitioners' understanding of concepts left undefined [36]; and (iii) the applicability of a provision to a concrete case requires expert assessment.

Despite these challenges and limitations [37], we argue that legal formalization is useful, especially when automation is desirable. First, formalization helps reduce ambiguity [57] and clarify what legal interpretation a system adheres to. Second, even where concepts are vague, techniques such as refinement can make mappings between legal and real-world facts explicit, contributing to greater clarity and transparency. Third, a formal encoding is needed to provide mathematical compliance guarantees and automation. Our work contributes to the legal formalization literature by providing further evidence for the practicality of formal approaches to CbD.

Formalization languages. Palmirani et al. developed LegalRuleML [5, 6, 63], a RuleML [14] extension tailored to legal rules. It is XML-based, which may hamper understandability. It relies on the *non-classical* modal defeasible logic [34] and does not support refinement. The LEGALEASE language [75] is less expressive than first-order logic, whereas Lex has MFOTL's expressivity. Being compositional, LEGALEASE cannot capture the typical structure of laws.

Van Binsbergen et al. introduced the eFLINT legal specification language [82] and report on preliminary experiments on legal compliance of government activities [84]. eFLINT describes the normative relations between actors (e.g., their duties) that arise from the law, rather than directly formalizing the rules in the law. It does not provide a framework for refining legal specifications into system specifications, support for enforcement through event causation, or formal end-to-end enforcement guarantees.

Merigoux et al. proposed Catala [48, 61], a 'programming language for the law' based on prioritized default logic [16]. Catala supports out-of-order definitions and exceptions, but does not natively encode events, obligations, or traces. As a result, it appears well suited to computation-heavy (e.g., tax) laws, but less so to CbD in real-time systems. Our IRC case study (Section 6) reproduces Merigoux et al.'s motivating example, showing that Lex is sufficiently expressive to encode such provisions.

Runtime enforcement in applications. Runtime enforcement [22, 31, 73] has been an active research area for over twenty years. Popular application domains include Android applications [23, 32, 67, 87] and IoT [21, 30, 77]. Moreover, general-purpose runtime enforcement tools have also been developed [40, 44, 64, 68], among which ENFGUARD provides state-of-the-art performance [45].

Privacy law formalization and enforcement. While several prior works have formalized privacy regulations [28, 29, 69, 70], such formalizations have rarely been used in Privacy by Design. To the best of our knowledge, only Robaldo et al. have developed a GDPR formalization with similar coverage to ours [69], but we are not aware of this formalization being used for automating compliance. Hublet et al. [41] have enforced simplified legal requirements using a runtime enforcement tool. In general, state-of-the-art PbD approaches [33, 41, 51–53] rely on ad hoc methodologies to enforce selected privacy requirements. They do not explicitly state their legal interpretation and cover only a limited portion of the law.

User studies of legal formalization. Bartolini et al. validated the formalization of two GDPR articles with legal experts [9, 10], but their formalization was later found to contain errors [43, 55]. Catala's authors [61] report on a user study with 18 law students. The students were given a presentation of the Catala language and asked to answer general questions about their understanding of a formalization of Section 121 IRC. Our validation methodology is more systematic, using a standardized usability test and comprehension questions with an interdisciplinary participant pool.

9 Conclusion

We have presented a formal theory, a methodology, and tools for rigorous Compliance by Design. Our main contribution is Lex, a language to formalize the law and refine it to system specifications.

We take inspiration from fields such as aerospace or hardware design, where formal methods are part of the standard validation

toolkit for critical systems. However, unlike in these fields, where most requirements admit mathematical definitions, legal compliance relies on experts' judgments about the soundness of interpretative steps. Accordingly, our methodology not only enforces complex specifications at runtime with formal guarantees but also supports the auditing of legal assumptions by legal and technical experts.

To evaluate our approach under real-world conditions, a more extensive user study involving both developers and external lawyers or regulators would be valuable. Additionally, we plan to integrate static verification techniques in our framework to further reduce runtime overhead and, potentially, broaden the scope of enforceable requirements. Finally, we envision that LLM-based tools could be used to automate and validate correct instrumentation [39, 54].

References

- [1] 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council. *Official Journal of the European Union* (2016).
- [2] Martin Abadi and Leslie Lamport. 1991. The existence of refinement mappings. *Theoretical Computer Science* 82, 2 (1991), 253–284.
- [3] Mohammad M Ahmadpanah, Daniel Hedin, and Andrei Sabelfeld. 2023. Lazytap: On-demand data minimization for trigger-action applications. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3079–3097.
- [4] Dominik Appelt. 2024. Von Privacy by Design zu Compliance by Design. *DuD: Datenschutz und Datensicherheit* 48, 11 (2024).
- [5] Tara Athan, Harold Boley, Guido Governatori, Monica Palmirani, Adrian Paschke, and Adam Wyner. 2013. Oasis LegalRuleML. In *ICAIL*. 3–12.
- [6] Tara Athan, Guido Governatori, Monica Palmirani, Adrian Paschke, and Adam Wyner. 2015. LegalRuleML: Design principles and foundations. *Reasoning Web. Web Logic Rules* (2015), 151–188.
- [7] Moshe Azar. 2007. Transforming Ambiguity into Vagueness in Legal Interpretation. *Interpretation, Law and the Construction of Meaning* (2007), 121–137.
- [8] Aaron Bangor, Philip Kortum, and James Miller. 2009. Determining what individual SUS scores mean: Adding an adjective rating scale. *JUS* 4, 3 (2009), 114–123.
- [9] Cesare Bartolini, Gabriele Lenzini, and Cristiana Santos. 2018. A legal validation of a formal representation of GDPR articles. In *JURIX*.
- [10] Cesare Bartolini, Gabriele Lenzini, and Cristiana Santos. 2019. An agile approach to validate a formal representation of the GDPR. In *JSAI-isAI*. Springer, 160–176.
- [11] David Basin, Søren Debois, and Thomas T Hildebrandt. 2016. In the nick of time: Proactive prevention of obligation violations. In *CSF. IEEE*, 120–134.
- [12] David Basin, Felix Klaedtke, Samuel Müller, and Eugen Zălinescu. 2015. Monitoring metric first-order temporal properties. *JACM* 62, 2 (2015), 1–45.
- [13] Guido Boella and Leendert Van Der Torre. 2003. Permissions and obligations in hierarchical normative systems. In *ICAIL*. 109–118.
- [14] Harold Boley, Said Tabet, and Gerd Wagner. 2001. Design rationale for RuleML: A markup language for semantic web rules. In *SWWS*, Vol. 1. 381–401.
- [15] Dino Bollinger, Karel Kubicek, Carlos Cotrini, and David Basin. 2022. Automating cookie consent and GDPR violation detection. In *USENIX Security*. 2893–2910.
- [16] Gerhard Brewka and Thomas Eiter. 2000. Prioritizing default logic. In *Intellectics and Computational Logic: Papers in Honor of Wolfgang Bibel*. Springer, 27–45.
- [17] John Brooke et al. 1996. SUS-A quick and dirty usability scale. *Usability evaluation in industry* 189, 194 (1996), 4–7.
- [18] Manfred Broy and Ketil Stølen. 2001. *Refinement*. Springer, New York, 241–252.
- [19] Pompeu Casanovas, Jorge González-Conejero, and Louis De Koker. 2018. Legal compliance by design (LCbD) and through design (LCtD): Preliminary survey. In *CEUR Workshop Proceedings*, Vol. 2049. 33–49.
- [20] Ann Cavoukian. 2011 [2009]. *Privacy by Design: The 7 Foundational Principles*. Information and Privacy Commissioner of Ontario.
- [21] Z. Berkay Celik, Gang Tan, and Patrick D. McDaniel. 2019. Iotguard: Dynamic enforcement of security and safety policy in commodity IoT. In *NDSS*.
- [22] Feng Chen and Grigore Roşu. 2003. Towards monitoring-oriented programming: A paradigm combining specification and implementation. *Electronic Notes in Theoretical Computer Science* 89, 2 (2003), 108–127.
- [23] Kevin Zhijie Chen, Noah M Johnson, Vijay D'Silva, Shuaifu Dai, Kyle MacNamara, Thomas R Magrino, Edward XueJun Wu, Martin C Rinard, and Dawn Xiaodong Song. 2013. Contextual policy enforcement in android applications with permission event graphs. In *NDSS*, Vol. 234.
- [24] Jan Chomicki. 1995. Efficient checking of temporal integrity constraints using bounded history encoding. *TODS* 20, 2 (1995), 149–186.
- [25] Michael R. Clarkson and Fred Schneider. 2010. Hyperproperties. *Journal of Computer Security* 18, 6 (2010), 1157–1210.
- [26] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *TACAS. Springer*, 337–340.
- [27] John F Decker. 2002. Addressing Vagueness, Ambiguity, and Other Uncertainty in American Criminal Laws. *Denv. UL Rev.* 80 (2002), 241.
- [28] Henry DeYoung, Deepak Garg, Limin Jia, Dilsun Kaynar, and Anupam Datta. 2010. Experiences in the logical specification of the HIPAA and GLBA privacy laws. In *ACM Workshop on Privacy in the Electronic Society*. 73–82.
- [29] Henry DeYoung, Deepak Garg, Dilsun Kaynar, and Anupam Datta. 2010. *Logical specification of the GLBA and HIPAA privacy laws*. Technical Report CMU-CyLab-10-007. Carnegie Mellon University.
- [30] Wenbo Ding, Hongxin Hu, and Long Cheng. 2021. Iotsafe: Enforcing safety and security policy with real IoT physical interaction discovery. In *NDSS*.
- [31] Úlfar Erlingsson. 2004. *The inlined reference monitor approach to security policy enforcement*. Ph. D. Dissertation.
- [32] Ylies Falcone, Sebastian Currea, and Mohamad Jaber. 2013. Runtime verification and enforcement for Android applications with RV-Droid. In *RV*. Springer, 88–95.
- [33] Mafalda Ferreira, Tiago Brito, José Fragoso Santos, and Nuno Santos. 2023. Rule-Keeper: GDPR-aware personal data compliance for web frameworks. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2817–2834.
- [34] Guido Governatori and Antonino Rotolo. 2008. A computational framework for institutional agency. *Artificial Intelligence and Law* 16, 1 (2008), 25–52.
- [35] Davide Grossi and Andrew Jones. 2013. Constitutive norms and counts-as conditionals. In *Handbook of deontic logic and normative systems*. College Publications, London, 407–441.
- [36] Clement Guitton, Simon Mayer, Aurelia Tamò-Larrieux, Kimberly Garcia, and Nicoletta Fornara. 2024. A proxy for assessing the automatic encodability of regulation. In *CS&Law*. 121–131.
- [37] Clement Guitton, Aurelia Tamò-Larrieux, and Simon Mayer. 2023. Mapping the Issues of Automated Legal Systems: Why Worry About Automatically Processable Regulation? *Artificial Intelligence and Law* 31, 3 (2023), 571–599.
- [38] H. L. A. Hart. 1994. *The Concept of Law* (2 ed.). Oxford University Press.
- [39] François Hublet, David Basin, Linda Hu, Srđan Krstić, and Lennard Reese. 2025. Instrumenting runtime enforcement. In *RV*. Springer, 160–180.
- [40] François Hublet, David Basin, and Srđan Krstić. 2022. Real-Time Policy Enforcement with Metric First-Order Temporal Logic. In *ESORICS*. 211–232.
- [41] François Hublet, David Basin, and Srđan Krstić. 2023. Enforcing the GDPR. In *ESORICS*. Springer, 400–422.
- [42] François Hublet, David Basin, and Srđan Krstić. 2024. User-controlled Privacy: Taint, Track, and Control. *PoPETS* 1 (2024).
- [43] François Hublet, Alexander Kvamme, and Srđan Krstić. 2024. Towards an Enforceable GDPR Specification. *arXiv:2402.17350 [cs.CR]*
- [44] François Hublet, Leonardo Lima, David Basin, Srđan Krstić, and Dmitriy Traytel. 2024. Proactive Real-Time First-Order Enforcement. In *CAV*.
- [45] François Hublet, Leonardo Lima, David Basin, Srđan Krstić, and Dmitriy Traytel. 2025. Scaling-Up Proactive First-Order Enforcement. In *CAV*.
- [46] François Hublet, Jakob Merane, Jonas Degelo, Wei-En Hsieh, Srđan Krstić, and David Basin. 2026. Lex: Turning Laws into Enforceable Security Policies (Extended Version). https://github.com/francois-hublet/lex/blob/main/extended_report/extended.pdf.
- [47] Sebastian Hunt and David Sands. 2008. Just forget it—the semantics and enforcement of information erasure. In *ESOP*. Springer, 239–253.
- [48] Liane Huttner and Denis Merigoux. 2022. Catala: moving towards the future of legal expert systems. *Artificial Intelligence and Law* (2022), 1–24.
- [49] ISO. 2023. *Consumer protection — Privacy by design for consumer goods and services*. Standard ISO 31700-1:2023. Geneva, Switzerland.
- [50] Klaus Julisch, Christophe Suter, Thomas Wotalla, and Olaf Zimmermann. 2011. Compliance by design—Bridging the chasm between auditors and IT architects. *Computers & Security* 30, 6-7 (2011), 410–426.
- [51] Felix Kalinowski, David Klein, Martin Johns, and Veelasha Moonsamy. 2025. Making Web Applications GDPR Compliant: A Comparative Evaluation of GDPR-Enforcement Frameworks. *PoPETS* (2025).
- [52] Farzane Karami, David Basin, and Einar Broch Johnsen. 2022. DPL: a language for GDPR enforcement. In *CSF. IEEE*, 112–129.
- [53] David Klein, Benny Rolle, Thomas Barber, Manuel Karl, and Martin Johns. 2023. General data protection runtime: Enforcing transparent GDPR compliance for existing applications. In *CCS*. 3343–3357.
- [54] Srđan Krstić, Hoang Nguyen, and David Basin. 2024. Model-driven Privacy. *PoPETS* 2024, 1 (2024), 314–329.
- [55] Alexander Kvamme. 2024. Bridging Logics: Transforming formalizations of GDPR into Enforceable MFOTL Specifications.
- [56] Erik Lagerspetz. 2017. *Institutional Facts*. Springer Netherlands, Dordrecht, 1–7.
- [57] Sarah B. Lawsky. 2016. Formalizing the code. *Tax L. Rev.* 70 (2016), 377.
- [58] Xavier Leroy. 2009. Formal verification of a realistic compiler. *CACM* 52, 7 (2009), 107–115.
- [59] libfuse project. 2015. libfuse: The reference implementation of the Linux FUSE (Filesystem in Userspace) interface. <https://github.com/libfuse/libfuse>.
- [60] Niels Lohmann. 2013. Compliance by design for artifact-centric business processes. *Information Systems* 38, 4 (2013), 606–618.

- [61] Denis Merigoux, Nicolas Chataing, and Jonathan Protzenko. 2021. Catala: a programming language for the law. *Proc. ACM Program. Lang.* 5, ICFP (2021).
- [62] Jakob Nielsen. 1993. *Usability engineering*. AP Professional.
- [63] Monica Palmirani, Guido Governatori, Antonino Rotolo, Said Tabet, Harold Boley, and Adrian Paschke. 2011. LegalRuleML: XML-based rules and norms. In *RuleML*. 298–312.
- [64] Srinivas Pinisetty, Yliès Falcone, Thierry Jéron, and Hervé Marchand. 2015. Tipex: A tool chain for timed property enforcement during execution. In *RV*. 306–320.
- [65] Ralf Poscher. 2011. Ambiguity and Vagueness in Legal Interpretation. In *Oxford Handbook of Language and Law*. Oxford University Press, 128–144.
- [66] Publications Office of the European Union. 2025. *Formex Version 4, Release 6.02.3*.
- [67] Siegfried Rasthofer, Steven Arzt, Enrico Lovat, and Eric Bodden. 2014. Droidforce: Enforcing complex, data-centric, system-wide policies in android. In *2014 Ninth International Conference on Availability, Reliability and Security*. IEEE, 40–49.
- [68] Matthieu Renard, Antoine Rollet, and Yliès Falcone. 2017. GREP: games for the runtime enforcement of properties. In *ICTSS*. Springer, 259–275.
- [69] Livio Robaldo, Cesare Bartolini, and Gabriele Lenzini. 2020. The DAPRECO knowledge base: representing the GDPR in LegalRuleML. In *LREC*. 5688–5697.
- [70] Livio Robaldo, Cesare Bartolini, Monica Palmirani, Arianna Rossi, Michele Martoni, and Gabriele Lenzini. 2020. Formalizing GDPR provisions in reified I/O logic: the DAPRECO knowledge base. *JLLI* 29 (2020), 401–449.
- [71] Shazia Sadiq and Guido Governatori. 2014. Managing regulatory compliance in business processes. In *Handbook on business process management 2: Strategic alignment, governance, people and culture*. Springer, 265–288.
- [72] Sanford Schane. 2002. Ambiguity and Misunderstanding in the Law. *T. Jefferson L. Rev.* 25 (2002), 167.
- [73] Fred Schneider. 2000. Enforceable security policies. *TISSEC* 3, 1 (2000), 30–50.
- [74] John R. Searle. 1969. *Speech acts: An essay in the philosophy of language*. CUP.
- [75] Shayak Sen, Saikat Guha, Anupam Datta, Sriram K. Rajamani, Janice Y. Tsai, and Jeannette M. Wing. 2014. Bootstrapping Privacy Compliance in Big Data Systems. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 327–342.
- [76] Marek J. Sergot, Fariba Sadri, Robert A. Kowalski, Frank Kriwaczek, Peter Hammond, and H Terese Cory. 1986. The British Nationality Act as a logic program. *CACM* 29, 5 (1986), 370–386.
- [77] Sabrina Sicari, Alessandra Rizzardi, Daniele Miorandi, Cinzia Cappiello, and Alberto Coen-Porisini. 2016. Security policy enforcement for networked smart objects. *Computer Networks* 108 (2016), 133–147.
- [78] Spiros Simitis, Gerrit Hornung, and Indra Spiecker gen. Döhmman (Eds.). 2024. *Datenschutzrecht: DS-GVO/BDSG* (2 ed.). Nomos, Baden-Baden. 2679 pages.
- [79] Scott Soames. 2012. Vagueness and the Law. In *The Routledge companion to philosophy of law*. Routledge, 95–108.
- [80] Christoph Sprenger, Tobias Klenze, Marco Eilers, Felix A Wolf, Peter Müller, Martin Clochard, and David Basin. 2020. Igloo: soundly linking compositional refinement and separation logic for distributed system verification. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 1–31.
- [81] Aurelia Tamò-Larrieux. 2018. *Designing for Privacy and Its Legal Framework*. Springer.
- [82] L Thomas Van Binsbergen, Lu-Chi Liu, Robert Van Doesburg, and Tom Van Engers. 2020. eFLINT: a domain-specific language for executable norm specifications. In *GPCE*. 124–136.
- [83] Tom Van Engers, Alexander Boer, Joost Breuker, André Valente, and Radboud Winkels. 2008. Ontologies in the legal domain. *Digital Government: E-Government Research, Case Studies, and Implementation* (2008), 233–261.
- [84] Nina Verheijen. 2024. Automating Compliance in Government Organisations using eFLINT. arXiv:2412.14183 [cs.CR]
- [85] Fabio Vitali and Monica Palmirani. 2017. Akoma Ntoso 3.0 Release Notes.
- [86] Jeremy Waldron. 2013. Vagueness in law and language: Some philosophical issues. In *Precedents, Statutes, and Analysis of Legal Concepts*. Routledge, 357–388.
- [87] Rubin Xu, Hassen Saidi, and Ross Anderson. 2012. Aurasium: Practical policy enforcement for android applications. In *USENIX Security*. 539–552.
- [88] Sheila Zingg, Srđan Krstić, Martin Raszyk, Joshua Schneider, and Dmitriy Traytel. 2022. Verified first-order monitoring with recursive rules. In *TACAS*. 236–253.

Acknowledgments

François Hublet was supported by the Swiss National Science Foundation grant “Model-driven Security & Privacy” (204796). David Basin was supported in part by the Novo Nordisk Foundation through the RECRUIT Sabbatical Grant NNF24OC0089993. We thank David Schild for its engineering support with the Lex artifact.

A Generative AI Usage

GPT-4.* and -5.* models were used only for minor type checking and rephrasing tasks in this paper. GPT, Claude Sonnet, and Claude

Opus models were used during software development.

B Open Science

All artifacts are available at <https://github.com/francois-hublet/lex>. The extended version of this paper [46], which contains all proofs and additional material, is available in the same repository.

C Ethical Considerations

A more complete version of this section can be found in our extended report [46].

Stakeholder Analysis. We identify the following stakeholders: individuals (at large) whose data is being processed by computer systems; companies processing individuals’ data; regulators; user study participants; and researchers.

Principles. Our research is anchored in a human-rights-based notion of privacy. Our user study was conducted according to our university’s ethical guidelines and approved by its Ethics Commission.

The participants received an information and consent form that presented the context of the study, what data was collected, and for what purpose. Participants were also allowed to leave the study at any stage without disadvantage, which none of them did. The experiment was conducted respectfully, with fair compensation.

Research was conducted under lawful employment or in the course of academic projects.

Harms. We identify the following potential harms: companies can be subject to additional costs if the development of new privacy technologies such as ours creates extensive obligations to adopt these technologies (e.g., per Art 25 GDPR); individuals participating in our user study may experience cognitive fatigue and similar psychological effects while completing our tasks; case study participants’ privacy may be compromised if personal data is released.

Mitigations. Additional costs incurred by companies if a new state-of-the-art privacy technology emerges are a deliberate consequence of current legislation and are offset by individual and rule-of-law benefits. Moreover, these costs are hypothetical as obligations to comply only emerge with the development of industrially viable solutions. Cognitive fatigue was mitigated through regular breaks.

To minimize privacy risks, the team managing the participant pool collected the name and e-mail address of the participants for recruitment and payment purposes only. We had no access to this data and only processed pseudonymized records with participants identified by random numbers—the participants did not enter their name in the system we used to collect their answers. The demographics collected (gender, with option not to disclose; age group; native language; self-assessed English proficiency level; field of study) are generic and do not allow us to re-identify participants.

Decision. The decision to conduct and submit this research was made based on the following considerations: (1) ethical individual and rule-of-law benefits largely offset the hypothetical harms in terms of compliance costs; (2) the user study was carefully designed and approved by our Ethics Committee to avoid excessive cognitive fatigue, exhaustion, and privacy breaches, and appropriately compensated; (3) authors conducted the research in line with applicable legal and ethical guidelines.